EVENTDECK: A UNIVERSITY EVENT TICKETING SYSTEM WITH TRANSACTIONAL SEAT RESERVATION AND QR CHECK-IN

PROJECT REPORT SUBMITTED TO

SRM INSTITUTE OF SCIENCE & TECHNOLOGY

IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE

AWARD OF THE DEGREE OF

MASTER OF COMPUTER APPLICATIONS

by

MATHESHWARAN R (RA2532241010311)
SUHAS C (RA2532241010312)
NITHYASHREE K (RA2532241010329)
DEVANANDA E (RA2532241010332)

UNDER THE GUIDANCE OF
Dr. V. NISHA ,M.Sc.,M.Phil.,PhD.,



DEPARTMENT OF COMPUTER APPLICATIONS

FACULTY OF SCIENCE AND HUMANITIES

SRM INSTITUTE OF SCIENCE & TECHNOLOGY

Kattankulathur – 603 203

Chennai, Tamil Nadu

OCTOBER – 2026

BONAFIDE CERTIFICATE

This is to certify that the project report titled "**EVENTDECK: A UNIVERSITY EVENT TICKETING SYSTEM WITH TRANSACTIONAL SEAT RESERVATION AND QR CHECK-IN**" is a Bonafide work carried out by

MATHESHWARAN R (RA2532241010311)
SUHAS C (RA2532241010312)
NITHYASHREE K (RA2532241010329)
DEVANANDA E (RA2532241010332)

under my supervision for the award of the Degree of Master of Computer Applications. To my knowledge the work reported herein is the original work done by the students.

Dr. V. Nisha
Assistant Professor
Department of Computer Applications
(GUIDE)

Dr. P. J. Arul Leena Rose
Professor & Head
Department of Computer Applications

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We thank the Almighty for the strength and the opportunity to complete this project, and our parents for their constant support and encouragement throughout our studies.

We are grateful to **Dr. T. R. Paarivendhar,** Chancellor, SRM Institute of Science and Technology, for providing the facilities and the academic environment in which this work was carried out.

We sincerely thank **Dr. A. Duraisamy,** Dean, Faculty of Science and Humanities, and Dr. S. Albert Antony Raj, Deputy Dean, Faculty of Science and Humanities, for their support and encouragement.

We express our thanks to **Dr. P. J. Arul Leena Rose,** Professor and Head, Department of Computer Applications, for her support in carrying out the project work and the reviews associated with it.

We are deeply thankful to our project guide, **Dr. V. Nisha,** Assistant Professor, Department of Computer Applications, for her guidance, timely suggestions and patience at every stage of this project.

We also thank all the faculty members of the Department of Computer Applications for their valuable comments during the reviews, and our friends and classmates who helped us in many small ways while the system was being built and tested.

MATHESHWARAN R
SUHAS C
NITHYASHREE K
DEVANANDA E

TABLE OF CONTENTS

1. INTRODUCTION 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives and Scope 2
1.3.1 Objectives 2
1.3.2 Scope of the Project 2
2. LITERATURE SURVEY 4
2.1 Event Ticketing as a Reservation Problem 4
2.2 Transactions and Concurrency Control 4
2.3 Access Control 5
2.4 Authentication, Sessions and Payment Security 5
2.5 Web Application Frameworks and Managed Services 6
2.6 Research Gap 6
Summary of the Literature Survey 6
3. SYSTEM ANALYSIS 7
3.1 Existing System 7
Limitations of the Existing System 7
3.2 Proposed System 7
3.3 Functional and Non-Functional Requirements 9
3.3.1 Functional Requirements 9
3.3.2 Non-Functional Requirements 10
3.4 Advantages of the Proposed System 11
4. SYSTEM ARCHITECTURE AND DESIGN 12
4.1 Overall Architecture 12
4.2 Technology Stack 13
4.3 Data Flow Diagram 14
4.4 Database Design 15
4.5 Module Design 17
4.6 Authentication and Session Design 18
4.7 Role-Based Authorisation and Event Moderators 19
4.8 User Interface Design 20
4.9 System Design Summary 21
5. EVENT AND TICKET MANAGEMENT 22
5.1 Accounts, Institutions and Campuses 22
5.2 Organizer Access and Event Creation 23
5.3 Audience Rules and the Per-Person Limit 24
5.4 Event Lifecycle 26

5.5 Event Discovery and Posters 27
5.6 Event Moderators 29
5.7 Summary 30
6. BOOKING PROCESSING AND DATA INTEGRITY 31
6.1 Eligibility of a Participant for an Event 31
6.2 The Booking Transaction 31
6.3 Seat Inventory and Reservation Holds 32
6.4 Booking States and Payments 33
6.5 Tickets and QR Codes 34
6.6 Check-in at the Venue 35
6.7 Database Constraints as a Safety Net 37
6.8 Audit Trail and Notifications 38
7. SYSTEM IMPLEMENTATION 40
7.1 Development Environment 40
7.2 Project Structure 40
7.3 Database Schema and Migrations 41
7.4 Authentication Implementation 42
7.5 Participant Interface 43
7.6 Organizer Interface 43
7.7 Administration, Cancellations and Refunds 45
7.8 Reports, Payouts and Exports 47
7.9 Configuration and Deployment Considerations 48
8. TESTING AND RESULTS 50
8.1 Testing Methodology 50
8.2 Test Environment 50
8.3 Test Suites and Overall Results 51
8.4 Functional Test Cases 52
8.5 Concurrency and Data Integrity Tests 53
8.6 Access Control and Isolation Tests 54
8.7 Static Checks and Interface Walkthrough 55
8.8 Limitations of the Testing Approach 57
9. CONCLUSION AND FUTURE ENHANCEMENTS 59
9.1 Conclusion 59
9.2 Future Enhancements 59
10. REFERENCES 61
11. APPENDICES 64
11.1 Selected Source Code 64
11.1.1 src/lib/server/bookings.ts — reserving and releasing seats 64
11.1.2 src/lib/domain/eligibility.ts — the audience rule 65

11.1.3 src/lib/server/checkin.ts — recording a check-in exactly once 65
11.2 Additional Screenshots 66
11.3 Automated Test Results 68
11.4 Demonstration Data Used for Screenshots 72
11.5 Glossary 73

ABSTRACT

EventDeck is a web-based event management and ticketing system for a multi-campus university. Built using Next.js, React, TypeScript and Supabase PostgreSQL, it allows students, faculty, alumni and visitors from other colleges to discover campus events, book free or paid tickets and receive QR-coded tickets on their phones. Organizers publish events with ticket tiers, seat limits, registration and check-in windows and audience rules that restrict an event to particular campuses, institutions or participant categories. The system reserves seats inside database transactions, holds unpaid seats for ten minutes, limits every person to four tickets per event and verifies Razorpay test-mode payments on the server. At the venue, organizers and the moderators they appoint scan tickets, and each ticket can be checked in only once even when several devices scan it at the same moment. Administrators review organizer applications, cancellations, refunds and payouts and export reports. EventDeck provides a secure, transparent and reliable approach to running university events from publication to attendance.

LIST OF TABLES

Table 3.1 User Roles and Responsibilities 8
Table 3.2 Functional Requirements 9
Table 3.3 Non-Functional Requirements 10
Table 4.1 Technology Stack 13
Table 4.2 Principal Database Tables 16
Table 4.3 Server Action and Route Handler Modules 17
Table 4.4 Summary of Role Permissions 20
Table 5.1 Audience Configurations Supported by an Event 25
Table 5.2 Event Statuses and Related States 26
Table 6.1 Conditions for a Participant to Book an Event 31
Table 6.2 Independent Status Fields of a Booking 34
Table 6.3 Principal Integrity Constraints 37
Table 7.1 Development and Test Environment 40
Table 7.2 Project Directory Structure 40
Table 7.3 Database Migrations 41
Table 7.4 Authentication Parameters 42
Table 7.5 Configuration Variables 48
Table 8.1 Test Environment for the Reported Run 50
Table 8.2 Automated Test Suites and Results 51
Table 8.3 Representative Functional Test Cases 52
Table 8.4 Concurrency and Integrity Tests (Historical Results) 53
Table 8.5 Access Control and Isolation Tests 54
Table 8.6 Seat Inventory Consistency Check on the Demonstration Database 55
Table 8.7 Defects and Observations 56
Table 8.8 Testing Not Executed 57
Table 11.1 Complete List of Automated Tests and Results 68
Table 11.2 Fictional Demonstration Data 72
Table 11.3 Glossary of Terms 73

LIST OF FIGURES

Figure 4.1 System Architecture of EventDeck 12
Figure 4.2 Level-1 Data Flow Diagram 15
Figure 4.3 Entity-Relationship Diagram of the Principal Tables 16
Figure 4.4 Request Authorisation Layers 19
Figure 4.5 Sign-in Page 21
Figure 5.1 Four-Step Account Creation Page 22
Figure 5.2 Create-Event Form: Basics, Schedule and Ticket Tiers 24
Figure 5.3 Access and Audience Rules in the Event Form 25
Figure 5.4 Event Status Lifecycle 26
Figure 5.5 Home Page with Poster Carousel 27
Figure 5.6 Event Listing Filtered by Category 28
Figure 5.7 Event Detail Page with Poster Banner and Ticket Selector 29
Figure 5.8 Team Tab for Assigning Event Moderators 30
Figure 6.1 Steps of the Booking Transaction 32
Figure 6.2 Booking Status Transitions 33
Figure 6.3 Booking Detail with QR Tickets and Progress 35
Figure 6.4 Server-Side Check-in Validation 36
Figure 6.5 Ticket Scanner Page 37
Figure 6.6 Event Activity Log 39
Figure 7.1 My Tickets Page of a Participant 43
Figure 7.2 Organizer Dashboard 44
Figure 7.3 Event Overview with Inventory by Tier 44
Figure 7.4 Attendance Page for an Event 45
Figure 7.5 Administrator Campus Overview 46
Figure 7.6 Cancellation Requests Awaiting Review 47
Figure 7.7 Reports Page with CSV Exports 48
Figure 11.1 People Page Filtered to Demonstration Accounts 66
Figure 11.2 Organizer Payouts and Settlements Page 67
Figure 11.3 Notifications Inbox of a Participant 67
Figure 11.4 Audit Log Viewed by an Administrator 68

1. INTRODUCTION

1.1 Background

A university campus hosts a steady stream of events: cultural festivals, technical symposia, hackathons, workshops, guest lectures, sports fixtures, club showcases and fresher's evenings. Each event has to be announced, its seats or passes distributed, payments collected where a fee applies, and attendees admitted at the door. The people who run these events are usually students and faculty members working for a club, a department or the Directorate of Student Affairs, and most of them organise events in addition to their regular studies or duties.

A large university is also not a single audience. SRM Institute of Science and Technology, for example, has students, faculty, staff and alumni across several campuses such as Kattankulathur, Ramapuram and Vadapalani, and many of its events are open to students of other colleges or to the public. Some events are meant only for one campus, some for every SRM campus, some for a list of invited colleges and some for anyone. The rules for who may register are therefore part of the event itself, and they need to be applied consistently.

Web technology now makes it practical for a university to run this process on its own system instead of spreading it across poster images, form links, payment screenshots and printed lists. EventDeck was developed to bring the whole life of an event — publication, registration, payment, ticket issue, check-in and settlement — into one application, with the rules enforced by the server and the database rather than by the people operating it.

1.2 Problem Statement

When campus events are managed with social-media posters, general online forms, shared spreadsheets and manual payment verification, several problems appear. Seat limits are not enforced when a form is submitted, so a popular event can receive more registrations than it has seats. Payment proofs are checked by hand and can be forged or reused. Eligibility rules such as “SRM KTR students only” depend on what applicants type into a form. Paper lists or screenshots used at the entrance can be copied, and nothing stops the same pass from being shown twice at two different gates. Organizers have no live view of how many seats remain, and when an event is cancelled there is no reliable record of who paid and must be refunded.

The problem addressed by this project can therefore be stated as follows: to design and implement a university event ticketing system that never issues more tickets than an event has seats, that applies audience and per-person limits on the server from trusted profile data, that verifies payments with the payment provider rather than with screenshots, that admits each ticket exactly once even when it is scanned by several moderators at the same time, and that keeps a complete, auditable record of bookings, cancellations, refunds and payouts.

1.3 Objectives and Scope

1.3.1 Objectives

The main objective of the project is to build a working event ticketing system for a multi-campus university and to verify that its central rules hold under realistic and adversarial use. The specific objectives are:

1. To let participants from SRM campuses, other colleges and the public create accounts with a verified institution profile, and to let club and department members apply for organizer access that an administrator approves.
2. To allow organizers to publish events with ticket tiers, seat capacities, registration and check-in windows, posters and audience rules covering institutions, SRM campuses, approved colleges and participant categories.
3. To process every booking as a single database transaction that reserves seats atomically, enforces a limit of four tickets per person and holds unpaid seats for ten minutes.
4. To verify Razorpay test-mode payments on the server, issue QR-coded tickets and PDF passes only for confirmed bookings, and recover or flag payments that arrive after a hold has expired.
5. To let organizers appoint event-specific moderators who can scan tickets at the venue, with each ticket accepted exactly once.
6. To give administrators tools for organizer applications, event cancellations, refunds, payment exceptions, payouts, audit logs and CSV reports.

1.3.2 Scope of the Project

The project covers the complete event workflow of a university: account creation and onboarding, organizer applications, event creation and publication, discovery and search, booking and payment, ticket issue, moderator management, check-in, cancellation and refunds, organizer payouts, notifications, audit logging and reporting. Three platform roles are supported —

participant, organizer and administrator — together with event moderators, who are ordinary accounts assigned to a single event.

Payments use Razorpay in Test Mode only; the system refuses live keys, so no real money is collected or refunded by this version. Payouts to organizers are recorded with a transfer reference entered by an administrator; the transfer itself happens outside the system. Waiting lists, invite-only allow-lists, seat maps with numbered seats and native mobile applications are outside the scope of this version.

2. LITERATURE SURVEY

This chapter reviews the published work, standards and technical documentation that informed the design of EventDeck. The survey is organised around the main concerns of the system: reserving a limited resource for many concurrent users, controlling who may do what, authenticating users and verifying payments safely, and the frameworks and managed services used to build the application.

2.1 Event Ticketing as a Reservation Problem

Selling tickets for an event with a fixed number of seats is a reservation problem of the same kind as airline and hotel booking, which Gray and Reuter use as a standard example of transaction processing [1]. Each booking must check that seats are available and take them in one indivisible step; otherwise two buyers who read the same remaining count can both succeed and the event is oversold. A second feature of ticketing is that payment happens after seats are chosen. Systems therefore place a temporary hold on the seats while the buyer pays and release it if payment does not complete, which turns a single decision into a small state machine with expiry.

Requests over the network can also be repeated: a user may double-click a button or a browser may retry a request after a timeout. The Idempotency-Key header proposed in the IETF HTTP APIs working group [22] describes the common remedy, in which the client sends a unique key with each logical operation and the server returns the original result for a repeated key instead of performing the operation twice.

2.2 Transactions and Concurrency Control

Codd's relational model [2] and the entity-relationship approach of Chen [3] provide the conceptual basis for the database design, and standard texts such as Silberschatz, Korth and Sudarshan [4] describe how relational systems implement transactions with the properties of atomicity, consistency, isolation and durability. Eswaran et al. [5] formalised serialisability and the role of locking in keeping concurrent transactions correct.

Berenson et al. [6] showed that the isolation levels defined by the SQL standard permit anomalies that are easy to overlook, such as lost updates and write skew, and Kleppmann [7] discusses how these anomalies appear in modern applications. PostgreSQL, the database used in this project, documents its row-level locks (SELECT … FOR UPDATE and FOR SHARE) [8], its

isolation levels [9] and the INSERT … ON CONFLICT clause [10], which lets a unique constraint decide atomically which of several concurrent inserts succeeds. EventDeck relies on these mechanisms rather than on checks made in application memory.

2.3 Access Control

Saltzer and Schroeder's principles for the protection of information [11] include least privilege, complete mediation and fail-safe defaults. Ferraiolo and Kuhn [12] and Sandhu et al. [13] developed role-based access control, in which permissions are attached to roles rather than to individual users. EventDeck uses three platform roles in this way, but roles alone are not sufficient: two organizers share the same role yet may manage only their own events, and a moderator may scan tickets for one event only. Permissions therefore also depend on the relationship between the user and the record being accessed.

PostgreSQL row-level security, exposed by Supabase as policies on each table [14], allows such relationship rules to be enforced inside the database as a second line of defence. The OWASP Top 10 [15] lists broken access control as the most serious web application risk, and the OWASP Application Security Verification Standard [16] recommends that every access decision be made on the server from trusted data.

2.4 Authentication, Sessions and Payment Security

NIST Special Publication 800-63B [17] gives current guidance on memorised secrets, favouring a reasonable minimum length over complex composition rules and requiring that passwords be stored only in a salted, hashed form. Supabase Auth, used by EventDeck, stores password hashes and issues JSON Web Tokens [18] that are kept in cookies and verified on the server for every request [27].

Two further sources shaped the ticket and payment design. RFC 4086 [19] explains why security tokens must come from a cryptographically secure random source; EventDeck's QR codes carry only such a token, and ISO/IEC 18004 [20] defines the QR symbology used to print it. Razorpay's documentation [21] requires that a payment be confirmed by verifying an HMAC signature on the server and that webhooks be verified against the exact request body. OWASP's guidance on CSV injection [23] informed the way exported reports neutralise spreadsheet formulas.

2.5 Web Application Frameworks and Managed Services

The project uses Next.js [24], a React-based framework [25] in which pages are rendered on the server and in which server functions, called Server Actions, can be invoked directly from forms. Database access is written with the Drizzle ORM [26], which produces typed SQL queries and versioned migrations. Supabase provides the managed PostgreSQL database together with authentication [27], realtime change notifications [28] and connection pooling [29]. Automated tests are written with Vitest [30] and browser tests with Playwright [31], and the interface design was checked against the W3C Web Content Accessibility Guidelines [32].

2.6 Research Gap

The sources reviewed above address the parts of the problem separately. The transaction-processing literature [1], [5]–[7] explains how to reserve a scarce resource without overselling; access-control work [11]–[13] explains how to separate the powers of different users; and payment and token standards [19]–[21] explain how to make a ticket that cannot be forged. General-purpose form and ticketing tools, on the other hand, are designed for a single organizer and a single audience and do not model a university with several campuses, external colleges and event-specific door staff.

For a university that wants to run its events on its own system, these ideas have to be combined in one application: seat reservation inside database transactions, audience rules evaluated from verified profiles, payment confirmation from the provider, tickets that are valid only when every independent condition holds, and check-in that remains correct when several moderators scan at once. EventDeck was built to provide this combination and to test it.

Summary of the Literature Survey

The survey established four design decisions that recur through the rest of this report: perform every seat change inside a database transaction with a conditional update; derive every permission and eligibility decision on the server from stored data; trust a payment only after server-side signature verification; and let unique constraints in the database decide races such as duplicate bookings and duplicate check-ins.

3. SYSTEM ANALYSIS

3.1 Existing System

Where a university has no dedicated system, events are commonly run with a combination of tools: a poster shared on social media, a general online form for registration, a payment link or a UPI number with a request to upload a screenshot, a spreadsheet exported from the form, and a printed or shared list used at the entrance. Each tool works on its own, but the rules of the event — the seat limit, who may attend, whether payment was received and whether a pass has already been used — are kept in people's heads.

Limitations of the Existing System

- Seat limits are not checked when a form is submitted, so more people can register than an event can hold.
- Payment is verified by looking at screenshots, which can be edited or reused for several registrations.
- Eligibility such as campus or college depends on what the applicant types, and is checked by hand, if at all.
- Passes shown at the entrance can be copied, and nothing prevents the same pass from being used twice at different gates.
- Organizers cannot see live counts of registrations, payments and attendance while an event is filling up.
- Cancellations and refunds are handled informally, with no reliable record of who paid and who has been refunded.
- Volunteers at the door need access to the whole response sheet, including personal details they do not need.

3.2 Proposed System

EventDeck replaces the poster-form-spreadsheet process with a single web application in which the rules of an event are enforced by the server and the database. An event is created once with its tiers, capacities, windows and audience rules. Bookings take seats in a transaction, payments are confirmed by Razorpay's signed callback and webhook, and every confirmed seat becomes a ticket with a random QR token. At the venue, the organizer or an appointed moderator scans the QR code and the server decides whether the ticket is valid and whether it has already been used.

One account can hold different responsibilities at different times. A student who books tickets as a participant can later become an organizer for a club and still book tickets for other events from the same account, and the same student can be appointed as a moderator for a particular event without receiving any other powers. The responsibilities are summarised in Table 3.1.

Table 3.1 User Roles and Responsibilities

Role	Main responsibilities
Participant	Creates an account with an institution profile; discovers and searches events; books free tickets, paid tickets or complimentary passes; pays through Razorpay; downloads QR tickets and PDF passes; cancels unpaid bookings; receives notifications; may apply for organizer access.
Organizer	Everything a participant can do, plus: creates, edits, publishes and unpublishes events; sets tiers, windows, audience rules and posters; reviews pass requests; appoints and removes moderators; scans tickets; views bookings, attendance and activity; exports CSV files; requests cancellation and payouts.
Event moderator	An ordinary account assigned by an organizer to one event. Can open the scanner for that event only and see the attendee name, tier and ticket number of each ticket scanned. Has no access to editing, bookings, payments or reports.
Administrator	Reviews organizer applications and changes account roles and status; approves or rejects cancellation requests or cancels events directly; processes refunds; resolves payment exceptions; processes payouts; reads the audit log; exports platform-wide reports. Administrator accounts are created only by a command-line script.

The proposed system follows the overall flow shown below. The detailed design of each part is presented in Chapters 4 to 7.

1. A participant creates an account and states their institution, campus and affiliation; club members apply for organizer access, which an administrator approves.
2. An organizer creates an event with tiers, capacities, windows and audience rules, uploads a poster and publishes it.
3. Participants find the event on the home page or in search and book tickets; free bookings are confirmed at once and paid bookings hold seats for ten minutes while the participant pays.

4. When payment is verified, tickets with QR codes are issued and appear in the participant's My tickets page.
5. On the event day, the organizer and the moderators scan tickets; each ticket is admitted once, and attendance is visible live.
6. If an event is cancelled, an administrator approves the cancellation, tickets are invalidated and refunds are queued; after the event the organizer may request a payout of the ticket revenue.

3.3 Functional and Non-Functional Requirements

3.3.1 Functional Requirements

The functional requirements identified for the system are listed in Table 3.2. Each requirement is implemented in the current version and is referred to by its identifier in later chapters.

Table 3.2 Functional Requirements

ID	Requirement
FR-01	Users shall sign up with name, e-mail and password and state their institution type, SRM campus or college, and affiliation; SRM students shall use an SRM e-mail address.
FR-02	Users shall sign in, sign out and reset a forgotten password by e-mail.
FR-03	A participant shall be able to apply for organizer access, and an administrator shall approve or reject the application.
FR-04	Organizers shall create, edit, publish and unpublish events with one or more ticket tiers, capacities, prices, windows, a poster and audience rules.
FR-05	Anyone shall be able to browse, search and filter published events by text, category and audience.
FR-06	Participants shall book free or paid tickets, or request a complimentary pass, up to four tickets per person per event.
FR-07	Paid bookings shall hold seats for ten minutes and be confirmed only after server-side verification of the Razorpay payment.
FR-08	Confirmed bookings shall receive one ticket per seat with a unique ticket number, a QR code and a downloadable PDF.
FR-09	Organizers shall approve or reject complimentary pass requests with a reason.
FR-10	Organizers shall appoint and remove up to ten moderators per event by e-mail address.

ID	Requirement
FR-11	Organizers and moderators shall check in tickets by camera or by typed ticket number, and each ticket shall be admitted only once.
FR-12	Organizers shall request cancellation of an event; administrators shall approve, reject or cancel directly, which invalidates tickets and queues refunds.
FR-13	Organizers shall register a bank or UPI account and request payouts of ticket revenue; administrators shall complete or reject them.
FR-14	The system shall notify users of booking, payment, approval, cancellation and refund events and keep an audit log of significant actions.
FR-15	Administrators and organizers shall export bookings, payments, attendance, refunds, people and audit data as CSV files.

3.3.2 Non-Functional Requirements

Table 3.3 Non-Functional Requirements

Quality	Requirement
Correctness	Seats shall never be over-allocated and a ticket shall never be checked in twice, including under concurrent requests.
Security	Roles, prices, eligibility, payment status and ticket validity shall be decided on the server from stored data; every table shall have row-level security.
Privacy	Moderators shall see only what they need at the door; one user shall never see another user's bookings or tickets.
Reliability	Repeated submissions, duplicate webhooks and late payments shall not create duplicate bookings or confirmations.
Usability	Pages shall work on phones and tablets, use plain language for errors and support keyboard navigation.
Performance	Common pages shall load within a few seconds on the hosted database; scanning shall return a result immediately.
Maintainability	Rules shall be implemented once in typed domain modules and covered by automated tests.

3.4 Advantages of the Proposed System

- Exact seat enforcement – seats are reserved with a conditional update inside a transaction, so an event cannot be oversold.
- Trustworthy payments – a booking is confirmed only after Razorpay's signature is verified on the server; screenshots play no part.
- Audience control – campus-only, inter-campus, inter-college and open events are enforced from verified profiles.
- Fair distribution – no person can hold more than four tickets for an event, across all of their bookings.
- Secure entry – each QR code carries only a random token, and the database admits each ticket once on any number of devices.
- Delegated door duty – moderators can scan for one event without seeing bookings, payments or personal data.
- Traceability – an audit log, notifications and status histories record who did what, and refunds and payouts are tracked to completion.

4. SYSTEM ARCHITECTURE AND DESIGN

4.1 Overall Architecture

EventDeck is a full-stack web application organised in four tiers, shown in Figure 4.1. Participants, organizers, moderators and administrators use the same application in a browser or on a phone. Every request first passes through the edge proxy in src/proxy.ts, which refreshes the Supabase session cookie and sends signed-out users away from protected areas. The application tier consists of React Server Components that render pages, Server Actions that handle form submissions, and route handlers for the scanner, payments, webhooks, ticket downloads, reports and the scheduled hold-expiry job.

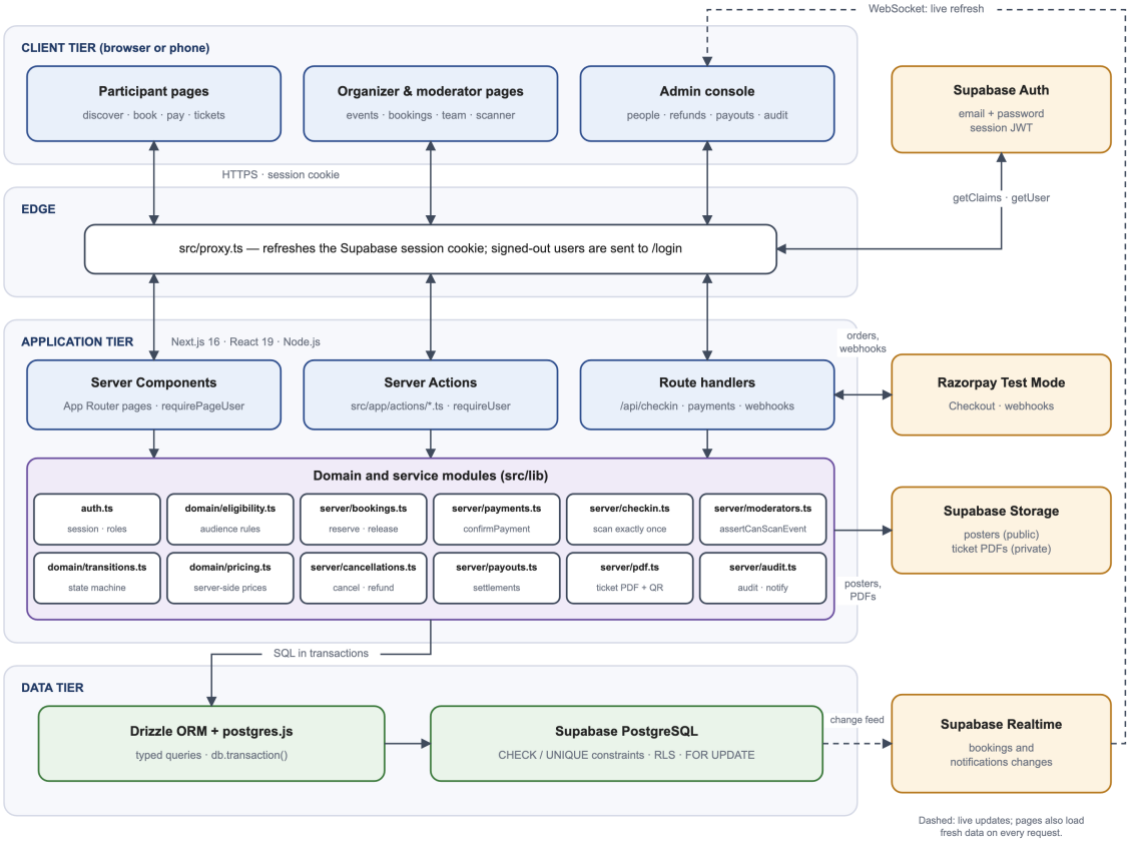


Figure 4.1 System Architecture of EventDeck

The business rules are kept in a small set of modules under src/lib. The most important are domain/eligibility.ts (audience rules), domain/pricing.ts (server-side prices), domain/transitions.ts (the booking state machine), server/bookings.ts (seat reservation and holds), server/payments.ts (payment confirmation), server/checkin.ts (scanning) and server/moderators.ts (who may scan).

Pages, actions and route handlers call these modules; none of them changes seat counts or statuses directly.

The data tier is a Supabase PostgreSQL database accessed through the Drizzle ORM and the postgres.js driver. Supabase also provides authentication, file storage for posters and ticket PDFs, and realtime change notifications that refresh a booking page and the notification bell as soon as the underlying rows change. Razorpay is called only from the server, and its webhooks are received by a dedicated route that verifies their signature.

4.2 Technology Stack

The technologies used in the project are listed in Table 4.1. Versions are those declared in the project's package.json file.

Table 4.1 Technology Stack

Layer	Technology (version)	Purpose in EventDeck
Framework	Next.js 16.4.0 (App Router, Turbopack)	Server-rendered pages, Server Actions, route handlers, edge proxy
UI library	React 19.3.0, TypeScript 5	Components and static typing across client and server
Styling	Tailwind CSS 4, shadcn/ui on Base UI 1.9	Responsive layout and accessible components
Forms and validation	React Hook Form 7.90, Zod 4.6	Client forms; the same schemas re-validate every input on the server
Database	Supabase PostgreSQL	Relational storage, constraints, row-level security
Data access	Drizzle ORM 0.45, drizzle-kit 0.31, postgres.js 3.4	Typed queries, transactions and SQL migrations
Authentication	Supabase Auth (supabase-js 2.117, @supabase/ssr 0.12)	E-mail and password accounts, session cookies, password reset
Storage and realtime	Supabase Storage and Realtime	Posters, cached ticket PDFs, live refresh of bookings and notifications
Payments	Razorpay (Test Mode)	Orders, Checkout, signed callbacks, webhooks and refunds

Layer	Technology (version)	Purpose in EventDeck
Tickets	qrcode 1.5, pdf-lib 1.17, html5-qrcode 2.3.8, sharp 0.35	QR images, PDF passes, camera scanning, poster compression
Testing	Vitest 5.0, Playwright 1.64	Unit, integration and end-to-end browser tests
Runtime	Node.js 24	Development and production server runtime

4.3 Data Flow Diagram

Figure 4.2 presents the level-1 data flow diagram. Five external entities interact with five processes. Participants sign up and book tickets; organizers publish events and receive booking and attendance information; Razorpay exchanges orders and payment events with the booking process; moderators send QR scans to the check-in process; and administrators make decisions about cancellations, refunds and payouts. The four data stores correspond to groups of tables described in the next section.

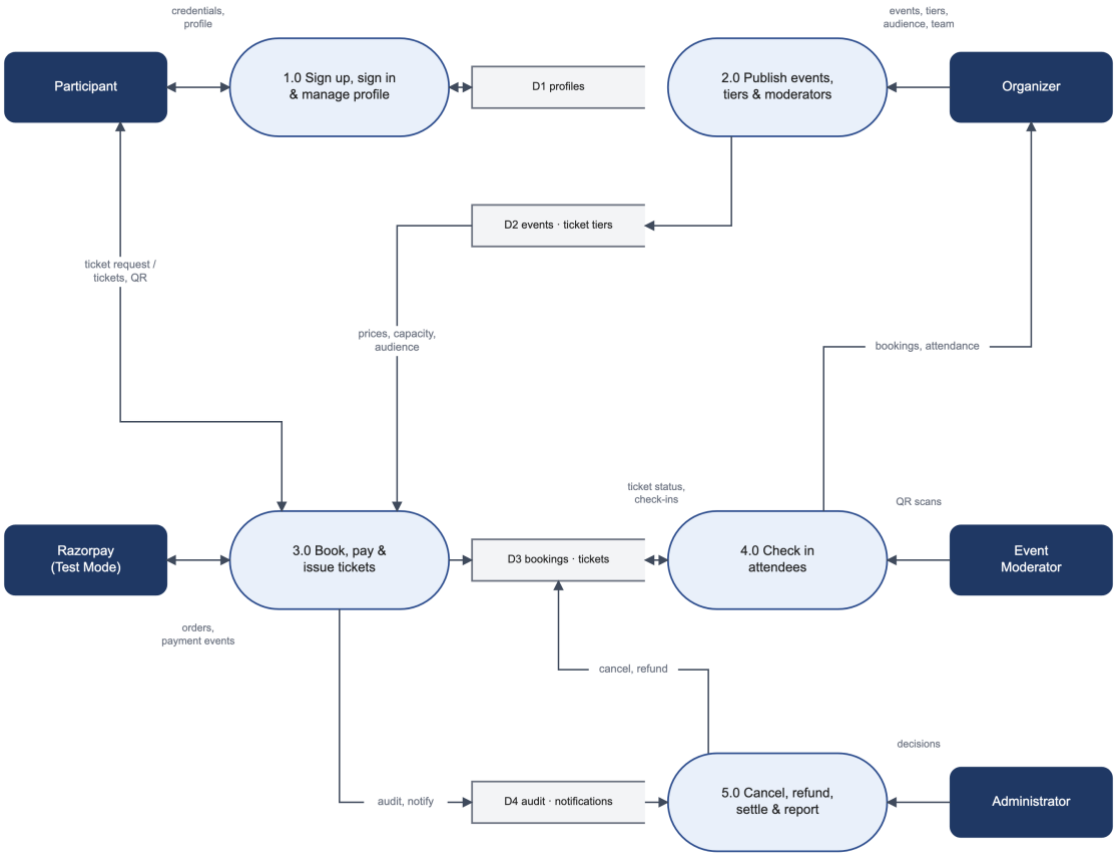


Figure 4.2 Level-1 Data Flow Diagram

4.4 Database Design

The database schema is defined in src/db/schema.ts. Figure 4.3 shows the principal tables and their relationships. A profile row is created for every Supabase Auth user and holds the role, the institution profile and an active flag. An event belongs to one organizer and has one or more ticket tiers; each tier records its capacity and the number of seats currently allocated. A booking belongs to one participant and one event and is broken down into booking items, one per tier. Each confirmed seat becomes a ticket, and a ticket can have at most one check-in.

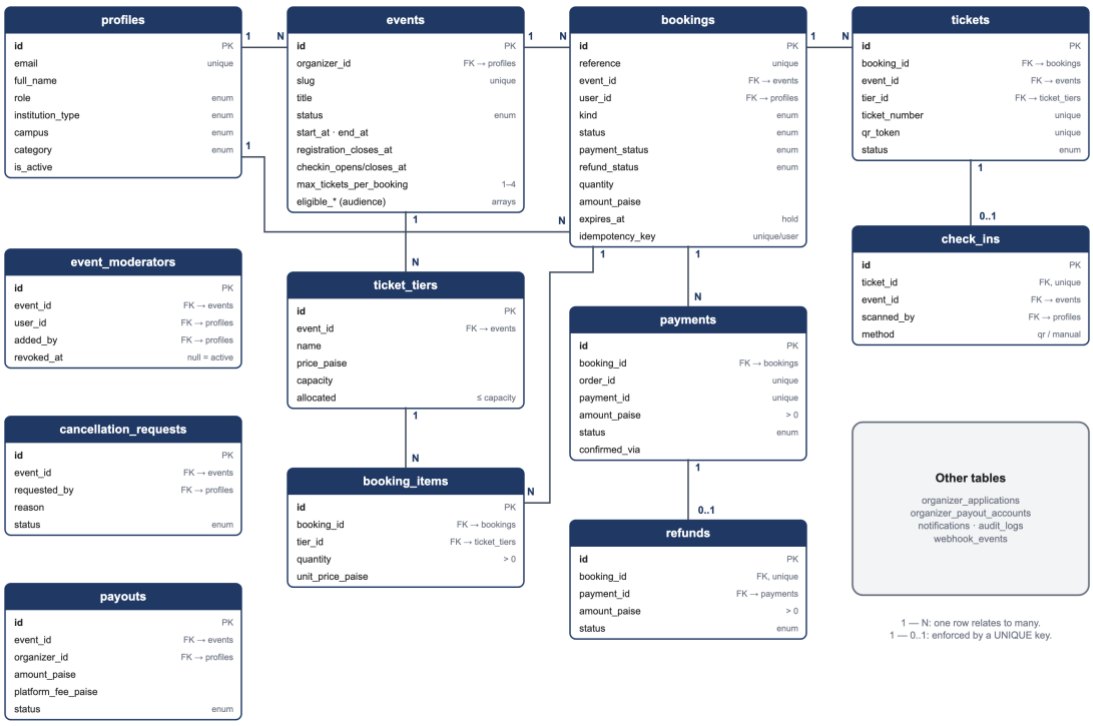


Figure 4.3 Entity-Relationship Diagram of the Principal Tables

Status values are stored as PostgreSQL enumerations, and the booking keeps its lifecycle status, its payment status and its refund status in separate columns so that, for example, a paid booking for a cancelled event can be shown as both cancelled and awaiting a refund. Table 4.2 summarises the purpose of each table.

Table 4.2 Principal Database Tables

Table	Purpose
profiles	One row per user: e-mail, name, role, institution type, campus, affiliation, category and active flag.
organizer_applications	Requests for organizer access, with the reviewing administrator's decision; one pending request per user.
events	Event details, schedule, windows, visibility, status, per-person limit, audience rules and a full-text search vector.
ticket_tiers	Tiers of an event with price, capacity and allocated seats; allocated may never exceed capacity.
bookings	A participant's order for an event: kind, status, payment and refund status, quantity, amount, hold expiry and idempotency key.

Table	Purpose
booking_items	Quantity and unit price per tier within a booking.
tickets	One per seat: ticket number, QR token, attendee name and validity.
check_ins	One row per admitted ticket, with the scanning user and method.
payments	Razorpay orders and payments for a booking, with verification details and flags.
webhook_events	Identifiers of processed webhook deliveries, used to ignore retries.
refunds	One refund per cancelled paid booking, with provider reference and status.
event_moderators	Moderator assignments per event; revoked assignments keep the revoking user and time.
cancellation_requests	Organizer requests to cancel an event and the administrator's review; one pending request per event.
organizer_payout_accounts, payouts	An organizer's bank or UPI details and payout requests with platform fee, net amount and transfer reference.
notifications, audit_logs	In-app notifications for users and a log of significant actions for administrators and organizers.

4.5 Module Design

The server-side logic that responds to users is divided into Server Action modules under src/app/actions and route handlers under src/app/api. Each module checks the caller, validates input with Zod and calls the domain and service modules described in Section 4.1. Table 4.3 lists them.

Table 4.3 Server Action and Route Handler Modules

Module	Responsibility
actions/auth.ts	Sign-in, four-step sign-up, sign-out, password reset request and reset.
actions/profile.ts	Profile updates and organizer applications.
actions/organizer.ts	Create, update, publish and unpublish events; request cancellation.
actions/bookings.ts	Create a booking, cancel an unpaid booking, approve or reject a pass request.
actions/moderators.ts	Add and revoke event moderators.

Module	Responsibility
actions/admin.ts	Applications, roles, account status, cancellations, refunds and payment exceptions.
actions/payouts.ts	Save payout accounts, request payouts and process them.
actions/notifications.ts	Fetch notifications and mark them as read.
api/payments/order, verify, failed	Create a Razorpay order from the booking amount, verify the Checkout signature, record failures.
api/webhooks/razorpay	Verify webhook signatures on the raw body and apply payment and refund events once.
api/checkin	Validate a scan and record the check-in.
api/tickets/[bookingId]	Generate or return the cached PDF pass for an authorised viewer.
api/reports/[type]	CSV exports for administrators and event organizers.
api/cron/expire-holds	Scheduled release of expired holds, protected by a secret.

4.6 Authentication and Session Design

Accounts are managed by Supabase Auth with e-mail and password. Passwords must have between 8 and 72 characters and contain at least one letter and one number, and they are stored by Supabase only as hashes. After sign-in, the session is kept in HTTP-only cookies managed by the @supabase/ssr library. The edge proxy refreshes these cookies when needed, and on the server getCurrentUser verifies the session with Supabase Auth on every request before loading the profile. A profile marked inactive by an administrator is treated as signed out, so deactivating an account takes effect on the next request.

The sign-up form never accepts a role. New accounts are created as participants by a database trigger, which also classifies the institution profile; the trigger accepts an SRM student classification only for an SRM e-mail domain (srmist.edu.in or srmuniv.ac.in), so a person cannot claim to be an SRM student by calling the authentication service directly. Password reset uses a one-time token sent by e-mail and exchanged at /auth/confirm, and the response to a reset request is the same whether or not the address has an account.

4.7 Role-Based Authorisation and Event Moderators

Authorisation is enforced in layers, shown in Figure 4.4. The edge proxy only redirects signed-out users. Pages call requirePageUser with the roles they allow, and Server Actions and route handlers call requireUser, which returns 401 or 403 instead of redirecting. Inside the service layer, assertCanManageEvent allows only the owning organizer or an administrator to act on an event, and assertCanScanEvent additionally allows an active moderator of that particular event. Finally, row-level security on all seventeen tables ensures that a user who calls the database API directly with their own token sees only their own rows.

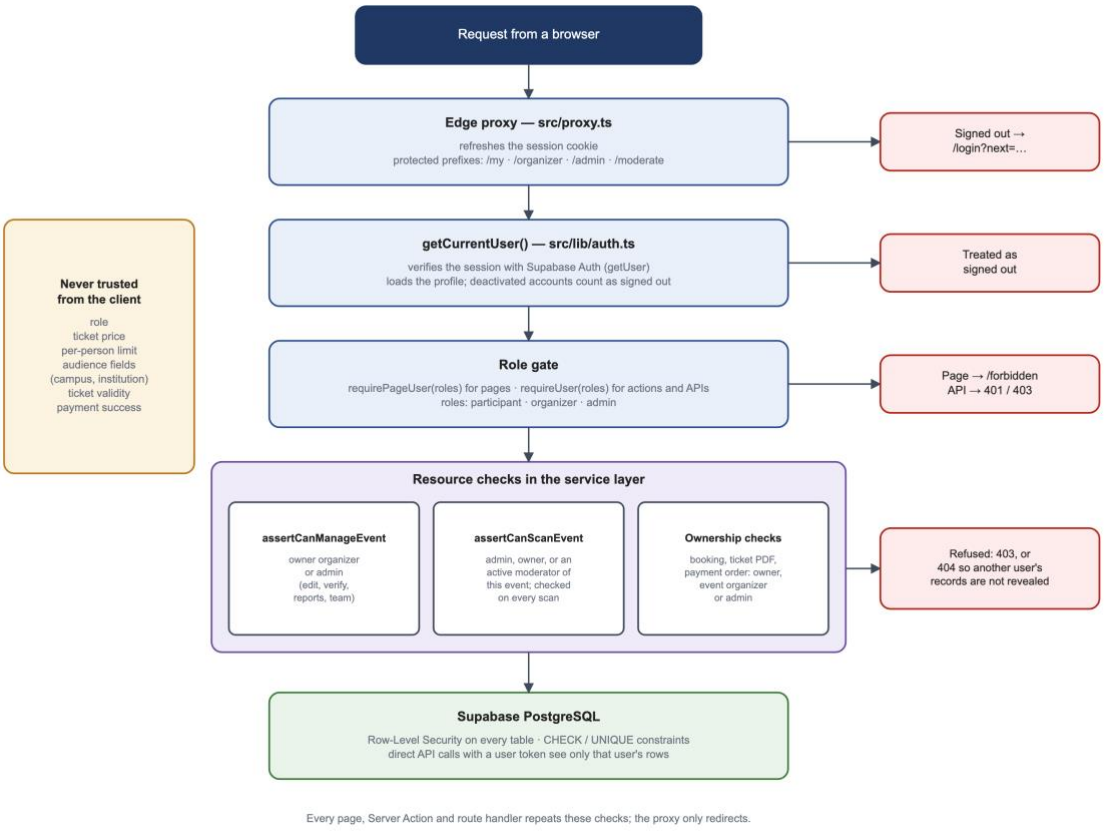


Figure 4.4 Request Authorisation Layers

Moderators are not a role. An organizer adds a moderator by e-mail address on the Team tab of an event, up to ten active moderators per event. The assignment is checked on every scan, so removing a moderator takes effect immediately, and a moderator who is also a participant keeps booking tickets as before. Table 4.4 summarises the permissions.

Table 4.4 Summary of Role Permissions

Capability	Admin	Organizer	Moderator	Participant
Browse events and book tickets	Yes	Yes	Yes	Yes
Create and edit events	All	Own	No	No
Approve or reject pass requests	All	Own	No	No
Add or remove moderators	All	Own	No	No
Scan tickets	All	Own	Assigned	No
View bookings and attendance	All	Own	No	Own only
Export CSV reports	All	Own events	No	No
Cancel events and process refunds	Yes	Request	No	No
Process payouts	Yes	Request	No	No
Change roles and account status	Yes	No	No	No

4.8 User Interface Design

The interface was designed around the tasks each role performs most often. The participant side resembles a consumer ticketing application: a home page with a poster carousel and horizontal rails of events, a searchable event list and an event page with a large poster and a ticket selector. Organizer and administrator pages share a dashboard layout with a sidebar, summary cards, tabs for each event and tables that can be exported. The sign-in page is shown in Figure 4.5.

Several principles guided the design. Statuses are always shown as labelled badges with consistent colours; error messages explain what to do next rather than repeating an error code; empty states explain how to fill them; and every page works at phone width, where the navigation collapses into a menu. A skip link and visible focus outlines support keyboard users.

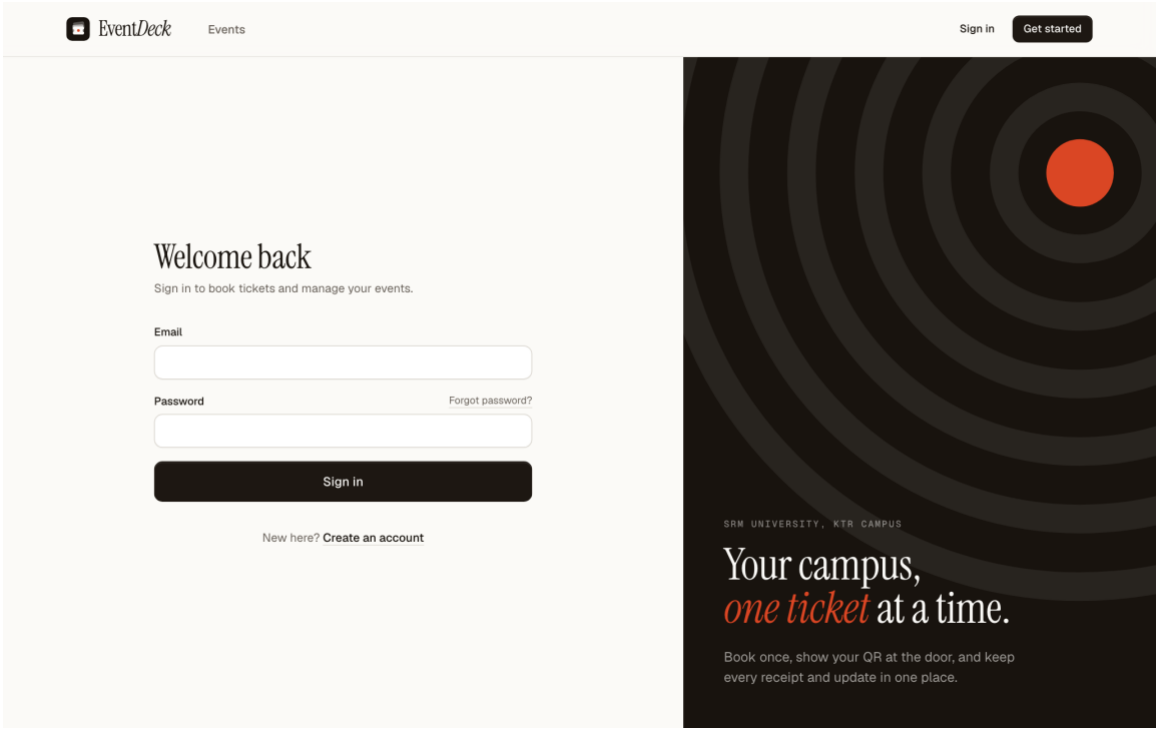


Figure 4.5 Sign-in Page

4.9 System Design Summary

The design separates concerns along clear lines. The edge proxy handles sessions, pages and actions authenticate and validate, domain modules hold the rules, and the database holds the final guarantees in the form of constraints, unique keys and row-level security. Because the important decisions — price, eligibility, seat availability, payment success and ticket validity — are made in one place on the server, the same rules apply whether a request comes from the interface, from a repeated request or from a direct call to an API.

5. EVENT AND TICKET MANAGEMENT

This chapter describes how accounts, organizers, events, audience rules, posters and moderators are set up and managed. The processing of bookings, payments and check-ins is described in Chapter 6.

5.1 Accounts, Institutions and Campuses

New users create an account through a four-step form (Figure 5.1): account details, origin, details and review. In the origin step a user chooses SRM University, another college or university, or guest. SRM users choose their campus — Kattankulathur, Ramapuram, Vadapalani or another SRM campus — and their affiliation as a student, faculty member, staff member or alumnus. Users from other colleges enter the name and city of their institution. SRM students must register with their university e-mail address, while SRM faculty, staff and alumni and external users may use a personal address.

From these answers the system derives a participant category — SRM student, student of another college, faculty, staff, alumnus or guest — which is used together with the institution type and campus when audience rules are checked. Users can edit their profile later, but the server refuses any change that would turn a non-SRM account into an SRM student.

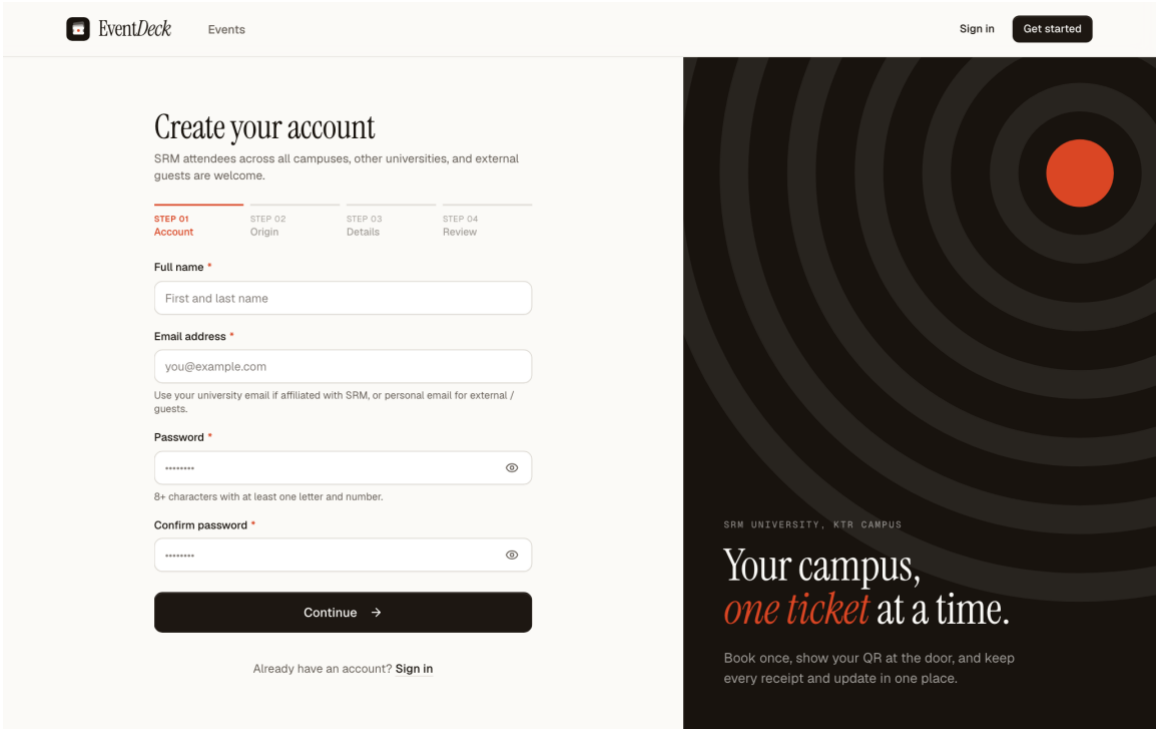


Figure 5.1 Four-Step Account Creation Page

5.2 Organizer Access and Event Creation

Any participant can apply for organizer access from their profile page by naming the club, department or society they represent and describing the events they will run. An administrator reviews the application and, on approval, the account's role becomes organizer. The organizer keeps their participant abilities, so the same account continues to book tickets for other events.

Events are created with the form shown in Figure 5.2. The Basics section holds the title, a one-line summary used on cards, the category and the venue. The Schedule section holds the start and end times, the time registration closes and the check-in window; all times are campus time (IST), and a helper suggests sensible windows from the start time. The Ticket tiers section lists one or more tiers, each with a name, price in rupees (zero for free tiers), capacity and optional perks. Capacity can be raised later but never lowered below the seats already taken. An event can be saved as a draft or published at once; publishing requires at least one tier and an end time in the future.

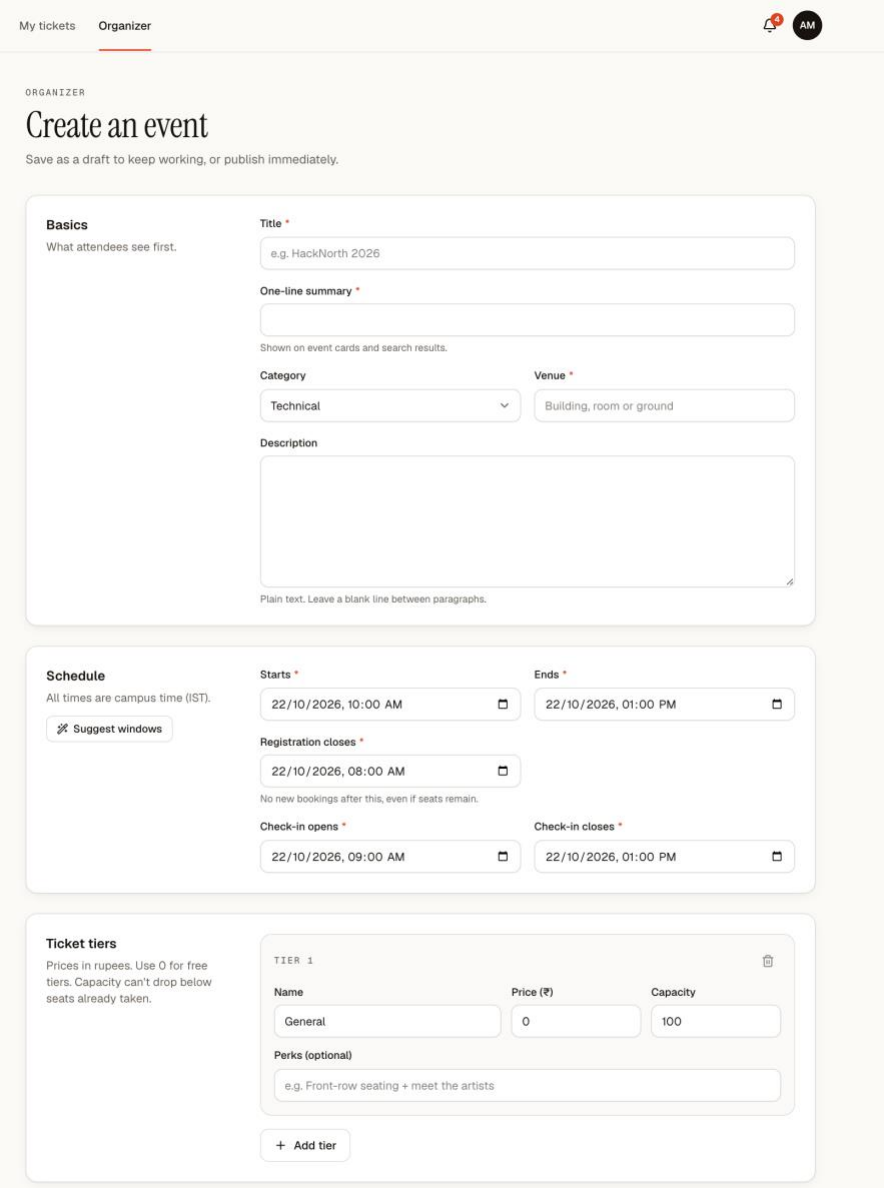


Figure 5.2 Create-Event Form: Basics, Schedule and Ticket Tiers

5.3 Audience Rules and the Per-Person Limit

The Access and rules section of the form (Figure 5.3) decides who may book. Visibility is either public, in which case the event is listed and searchable, or unlisted, in which case it is reachable only by its link. The organizer then ticks the institution types allowed, the SRM campuses allowed, an optional list of approved colleges and the participant categories allowed, and chooses whether complimentary pass requests are accepted. The combinations produce the four kinds of event described in Table 5.1.

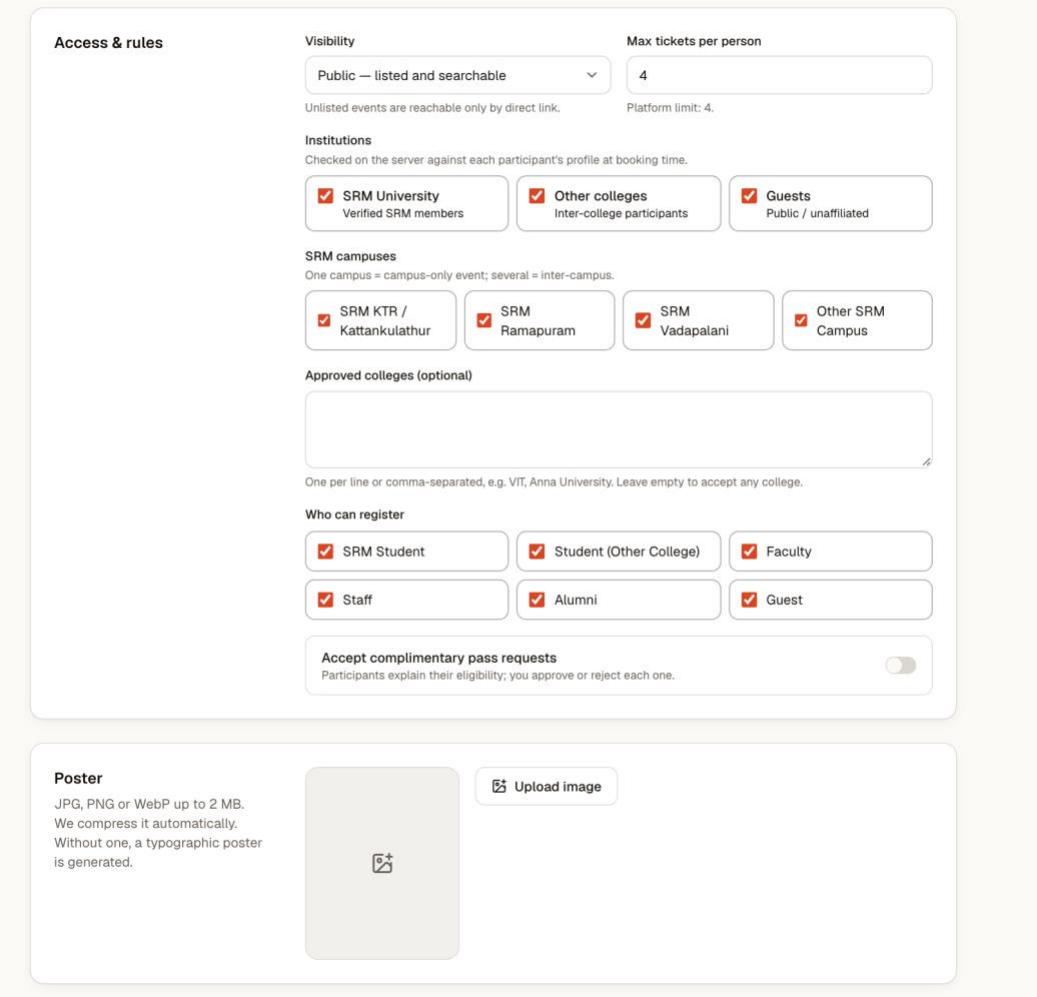


Figure 5.3 Access and Audience Rules in the Event Form

Table 5.1 Audience Configurations Supported by an Event

Kind of event	Configuration	Label shown to users
Campus-only	Institution type SRM; one SRM campus ticked	SRM <campus> only
Inter-campus	Institution type SRM; several SRM campuses ticked	SRM only
Inter-college	SRM and other colleges allowed; optional list of approved colleges (up to 25), matched ignoring case, spaces and punctuation	Inter-college
Open	All institution types, campuses and categories allowed	Open to all

Independently of the audience, every event has a maximum number of tickets per person. The organizer may choose any value from one to four; four is a platform-wide limit enforced in the

event form, in the server-side validation, in the booking transaction and by a CHECK constraint on the events table. The limit applies to the total number of tickets a person holds across all of their live bookings for the event, not only to a single booking.

5.4 Event Lifecycle

Figure 5.4 shows the life of an event. An event is created as a draft or published directly. A published event can be unpublished back to draft by its organizer or by an administrator, in which case the organizer is notified with the administrator's note. Organizers cannot cancel an event themselves: they submit a cancellation request with a reason, and an administrator approves or rejects it. Administrators can also cancel an event directly. Cancellation is final, and in the same transaction live bookings are moved to event_cancelled, tickets are invalidated, refunds are queued for paid bookings and every affected participant is notified. Table 5.2 lists the statuses.

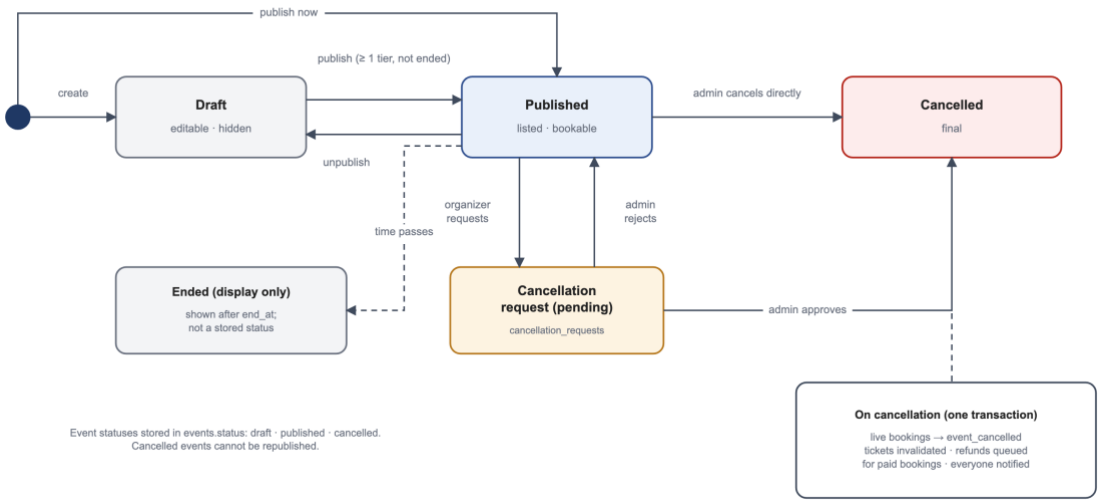


Figure 5.4 Event Status Lifecycle

Table 5.2 Event Statuses and Related States

Status / state	Meaning	Set by
draft	Editable and hidden from discovery; cannot be booked.	Organizer (create, unpublish); administrator (unpublish)
published	Listed (if public) and bookable until registration closes.	Organizer or administrator (publish)

Status / state	Meaning	Set by
cancelled	Final; tickets void, refunds queued; cannot be republished.	Administrator (approve request or cancel directly)
Cancellation request: pending	Waiting for an administrator; the event remains as it was.	Organizer
Cancellation request: approved / rejected	Recorded with the reviewer and note.	Administrator
Ended	Shown after the end time; not a stored status.	Derived from end_at

5.5 Event Discovery and Posters

The home page (Figure 5.5) opens with a carousel of upcoming events that have posters, followed by category chips and horizontal rails: featured events on campus, events open to all colleges and one rail per category. The event list (Figure 5.6) offers text search, implemented with PostgreSQL full-text search using prefix matching, together with category and audience filters, and shows how many tickets remain when an event is nearly full.

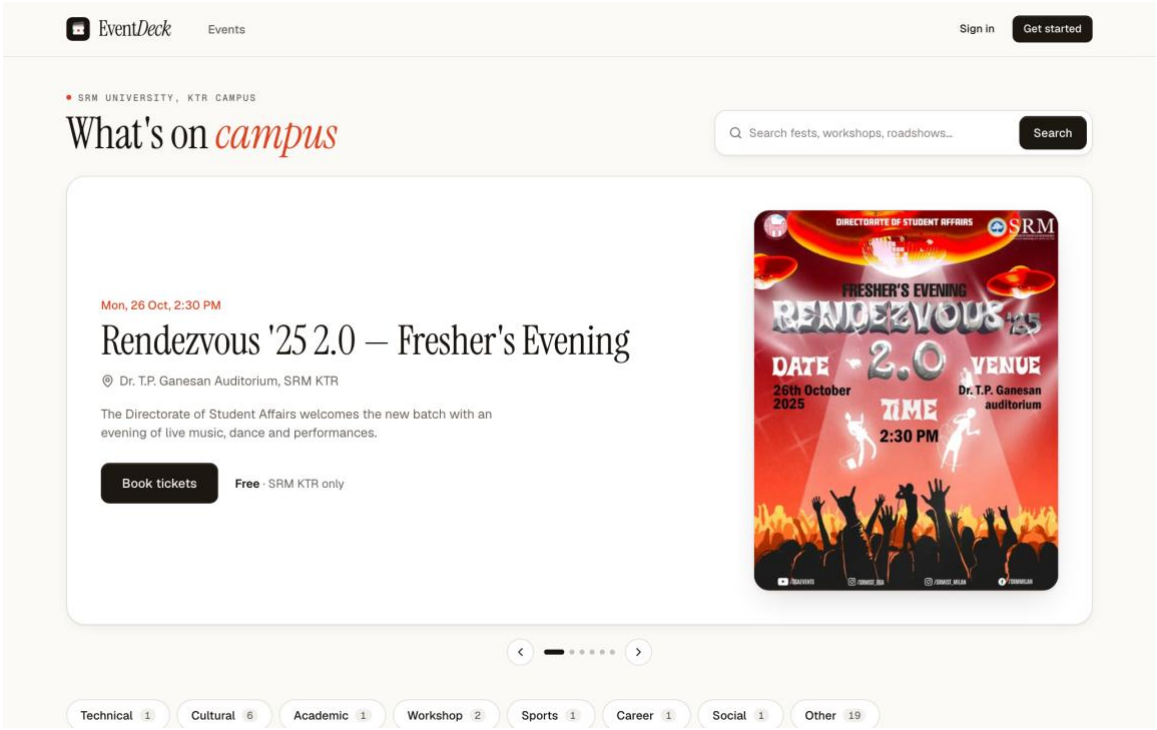


Figure 5.5 Home Page with Poster Carousel

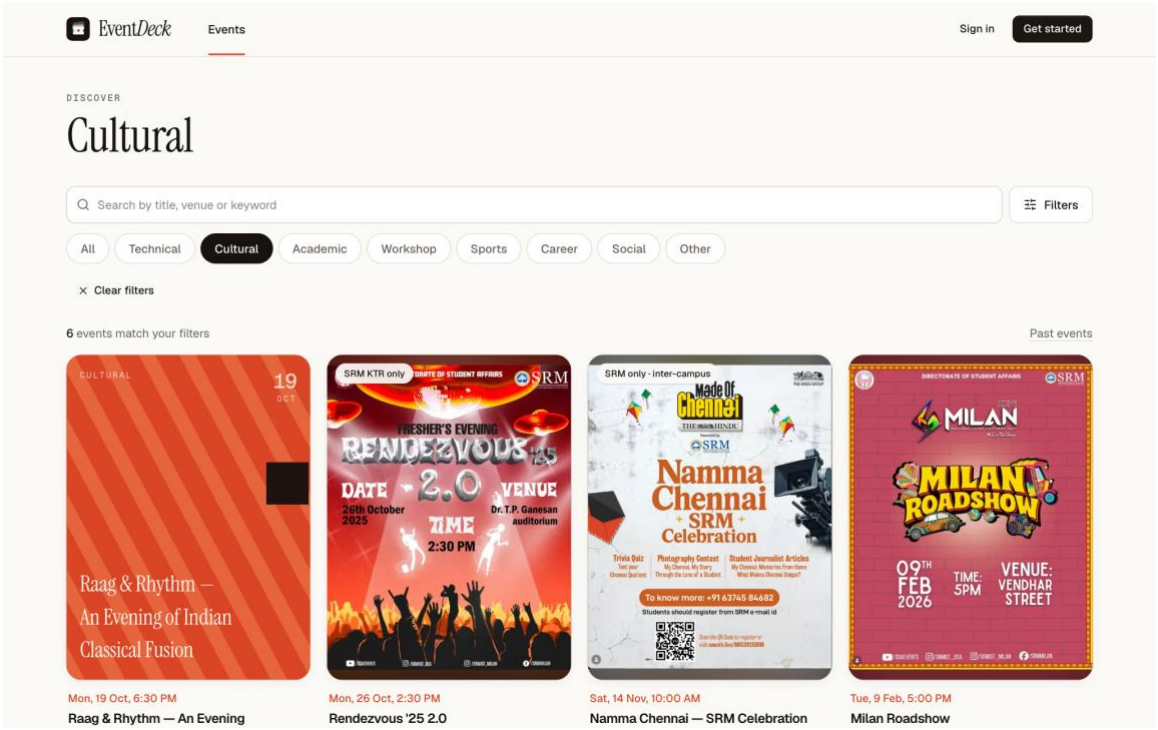


Figure 5.6 Event Listing Filtered by Category

Organizers may upload a JPEG, PNG or WebP poster of up to 2 MB. The server checks the file type from its first bytes rather than from its name, converts it to WebP no wider than 1,600 pixels and stores it in a public bucket; events without a poster are shown with a generated typographic design. On the event page (Figure 5.7) the poster is shown in full over a blurred copy of itself, so posters of any shape are displayed without cropping, and the ticket selector shows each tier, its price and the per-person limit.

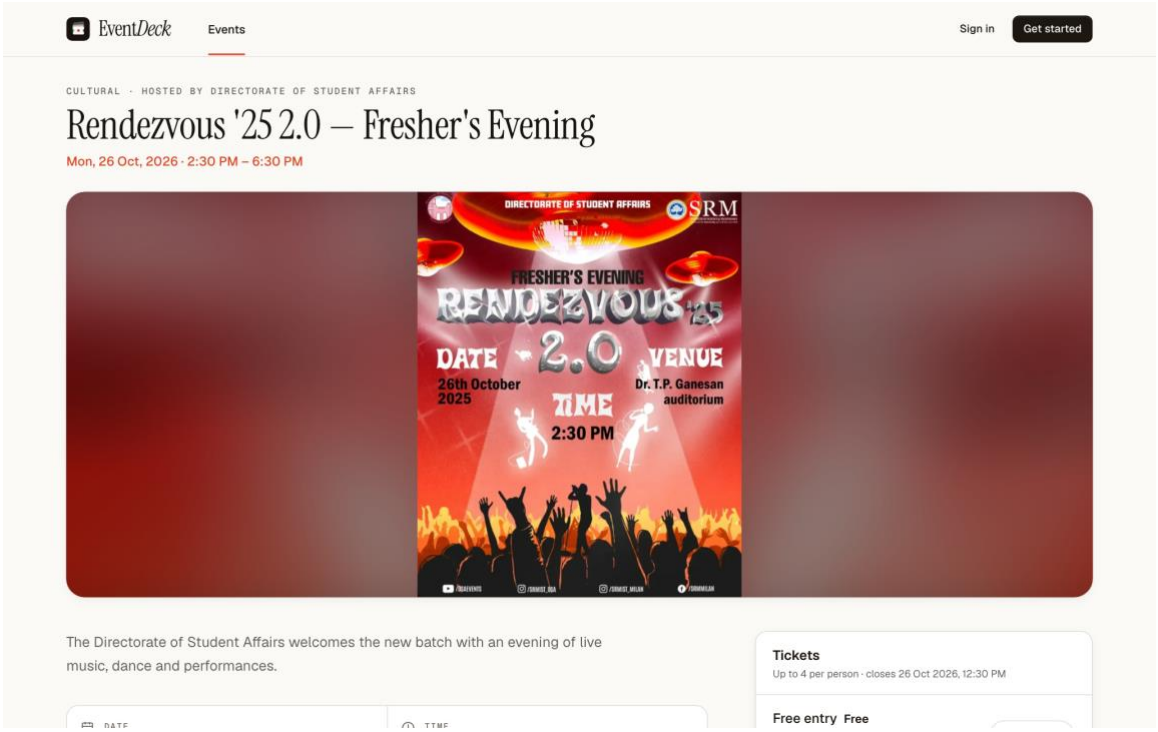


Figure 5.7 Event Detail Page with Poster Banner and Ticket Selector

5.6 Event Moderators

Large events need several people at the entrance. On the Team tab of an event (Figure 5.8) the organizer enters the e-mail address of an existing EventDeck account and adds it as a moderator. The system refuses unknown addresses, duplicates, the organizer's own address and an eleventh active moderator, and it notifies the person who was added. The moderator then sees a Check-in item in their menu, which lists only the events they moderate and opens the same scanner the organizer uses.

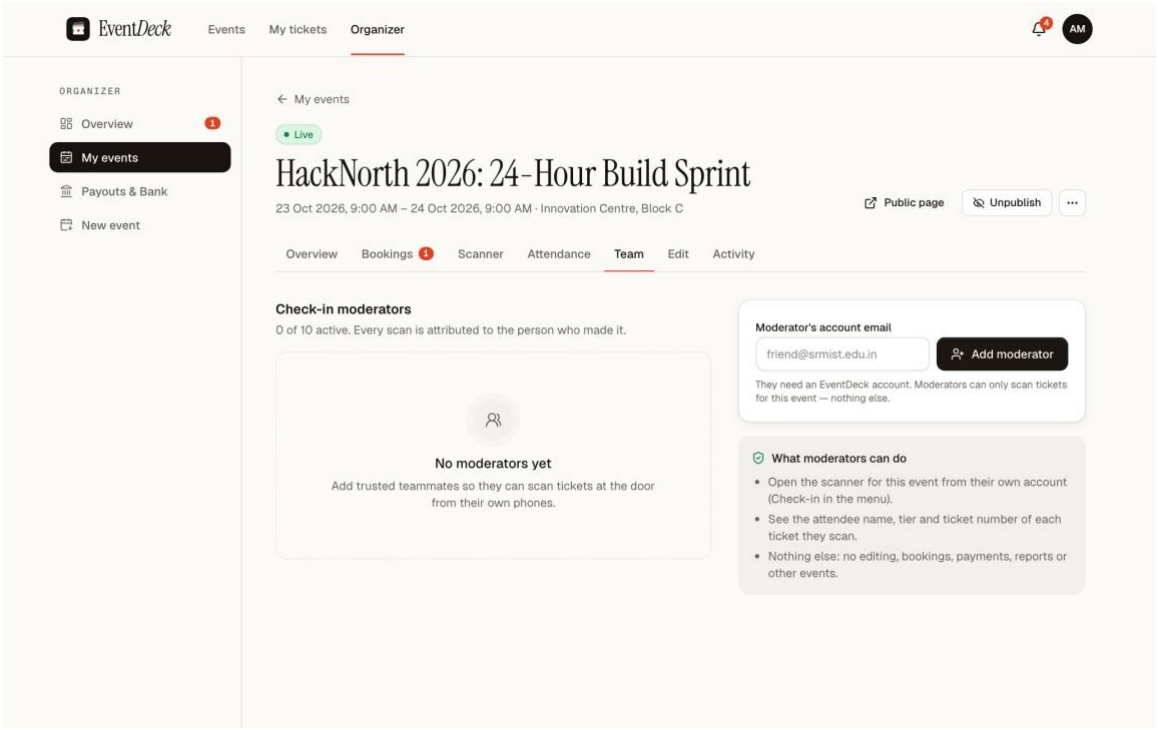


Figure 5.8 Team Tab for Assigning Event Moderators

Every check-in records the account that scanned it, so the attendance list and the CSV export show which moderator admitted each attendee. Removing a moderator marks the assignment as revoked with the time and the user who revoked it; the next scan by that account is refused.

5.7 Summary

Setting up an event in EventDeck therefore involves a small number of well-defined steps: the organizer obtains organizer access, creates the event with tiers, windows and audience rules, uploads a poster, publishes it and appoints moderators. From that point the rules are applied automatically to every booking and every scan, as described in the next chapter.

6. BOOKING PROCESSING AND DATA INTEGRITY

This chapter explains how a booking is processed, how seats are accounted for, how payments are confirmed, how tickets are checked in and how the system keeps its data consistent when many requests arrive at the same time.

6.1 Eligibility of a Participant for an Event

Before a booking is created, the function checkEligibility in src/lib/domain/eligibility.ts decides whether the event is meant for the participant. It reads only the participant's stored profile and the event's stored rules; any institution, campus, category or user identifier included in the request is ignored. The conditions are listed in Table 6.1, and a refusal tells the participant which condition failed, for example that the event is only for SRM Kattankulathur.

Table 6.1 Conditions for a Participant to Book an Event

Condition	How it is checked
Event is open	Status is published (not draft or cancelled) and the current time is before registration_closes_at.
Institution type	The profile's institution type is in eligible_institution_types.
SRM campus	For SRM users, the profile's campus is in eligible_campuses.
Approved college	If the list is not empty, the normalised college name of an external user is in eligible_institutions.
Participant category	The derived category is in eligible_categories.
Pass requests	Complimentary requests are accepted only if allow_pass_requests is set.
Per-person limit	Tickets already held in live bookings plus the new quantity do not exceed min(event limit, 4).

6.2 The Booking Transaction

Figure 6.1 shows the steps of createBooking in src/lib/server/bookings.ts. Two inexpensive steps are performed first, without taking any locks: if the same participant has already submitted a booking with the same idempotency key, that booking is returned; and any holds for the event whose ten minutes have passed are released. The rest of the work happens in one database transaction.

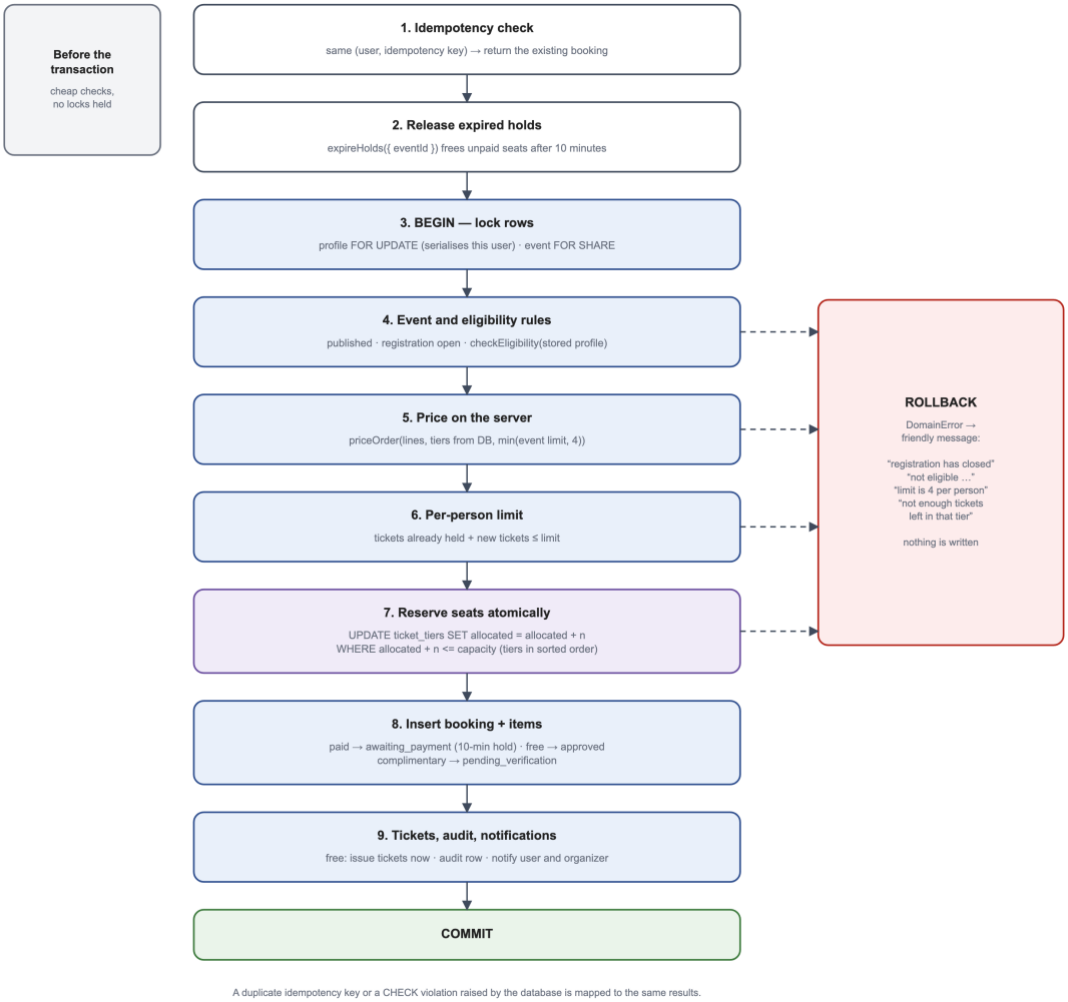


Figure 6.1 Steps of the Booking Transaction

Inside the transaction the participant's profile row is locked with FOR UPDATE, which serialises the bookings of one person and makes the per-person limit exact, and the event row is locked with FOR SHARE, so that an administrator cannot cancel the event half-way through a booking. The event rules and eligibility are checked, the order is priced from the tier rows in the database, and the number of tickets the participant already holds is counted. Seats are then reserved, the booking and its items are inserted, and for free bookings the tickets are issued at once. If any step fails, the transaction is rolled back and nothing is written; the participant sees a plain message such as “Sorry — not enough tickets left in that tier.”

6.3 Seat Inventory and Reservation Holds

Seats are taken and returned only through two functions, reserveInventory and releaseInventory, and always inside the same transaction as the booking status change that justifies

them. reserveInventory performs, for each tier, a single conditional statement: UPDATE ticket_tiers SET allocated = allocated + n WHERE id = … AND allocated + n <= capacity. If no row is updated, the tier does not have enough seats and the booking fails. Because the check and the increment are one statement, two concurrent bookings cannot both take the last seat. Tiers are always updated in the same sorted order, which prevents two multi-tier bookings from deadlocking.

A paid booking is created in the awaiting_payment status with an expiry time ten minutes ahead. The function expireHolds releases bookings whose time has passed; it is called before every booking, when participants open their bookings and from a scheduled job. Each release is itself a compare-and-set update that only succeeds while the booking is still awaiting payment, so a hold is released exactly once even if several requests try to expire it at the same moment, and never after payment has been recorded.

6.4 Booking States and Payments

The allowed booking statuses and transitions are defined in src/lib/domain/transitions.ts and shown in Figure 6.2. Every status change is made through transitionBooking, which first checks that the move is allowed and then runs UPDATE … WHERE status IN (expected statuses); if another request has already moved the booking, the update affects no row and the change is abandoned.

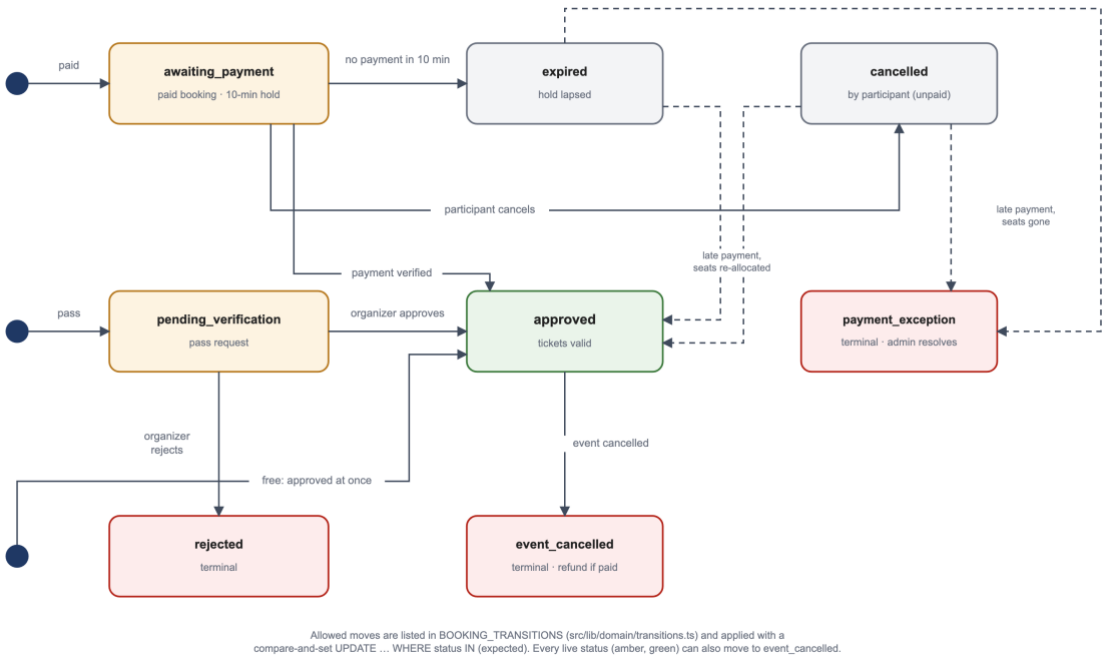


Figure 6.2 Booking Status Transitions

Free bookings are approved immediately. Complimentary pass requests wait in pending_verification until the organizer approves or rejects them with a reason. For paid bookings, the server creates a Razorpay order whose amount is read from the booking row, never from the browser. When the participant pays, the Checkout callback and the payment webhook each verify Razorpay's HMAC signature and then call confirmPayment, which locks the payment and booking rows, approves the booking, issues the tickets and notifies the participant and the organizer. Whichever arrives second finds the payment already recorded and does nothing. If a payment arrives after the hold has expired, the system tries to take the same seats again; if they are gone, the booking becomes a payment exception for an administrator to resolve, and no ticket is issued. Table 6.2 lists the independent status fields.

Table 6.2 Independent Status Fields of a Booking

Field	Values	Meaning
status	awaiting_payment, pending_verification, approved, rejected, cancelled, expired, event_cancelled, payment_exception	Where the booking is in its lifecycle
payment_status	not_required, pending, paid, failed, expired	Outcome of payment, independent of approval
refund_status	not_applicable, pending, processed, failed	Progress of a refund after cancellation
ticket status	valid, invalidated	Per ticket; invalidated tickets keep the reason

6.5 Tickets and QR Codes

When a booking is approved, issueTickets creates one ticket per seat. Each ticket has a readable ticket number made from a short code for the event and two random groups, such as RRAE-BVZQ-W8PL, and a QR token of 192 random bits from the operating system's secure random source. The QR code contains only the text EVD1: followed by this token; it does not contain the participant's name, the event or any status. Ticket creation is idempotent because the pair of booking and seat number is unique.

Figure 6.3 shows the booking page a participant sees. Each ticket shows the attendee name, tier, ticket number, QR code and validity, and a progress list shows when seats were reserved,

payment verified, the booking approved and tickets issued. The page refreshes automatically through Supabase Realtime when the booking changes. The Download PDF button produces a PDF pass with one page per ticket, generated with pdf-lib and cached in a private storage bucket; only the booking's owner, the event's organizer or an administrator can download it.

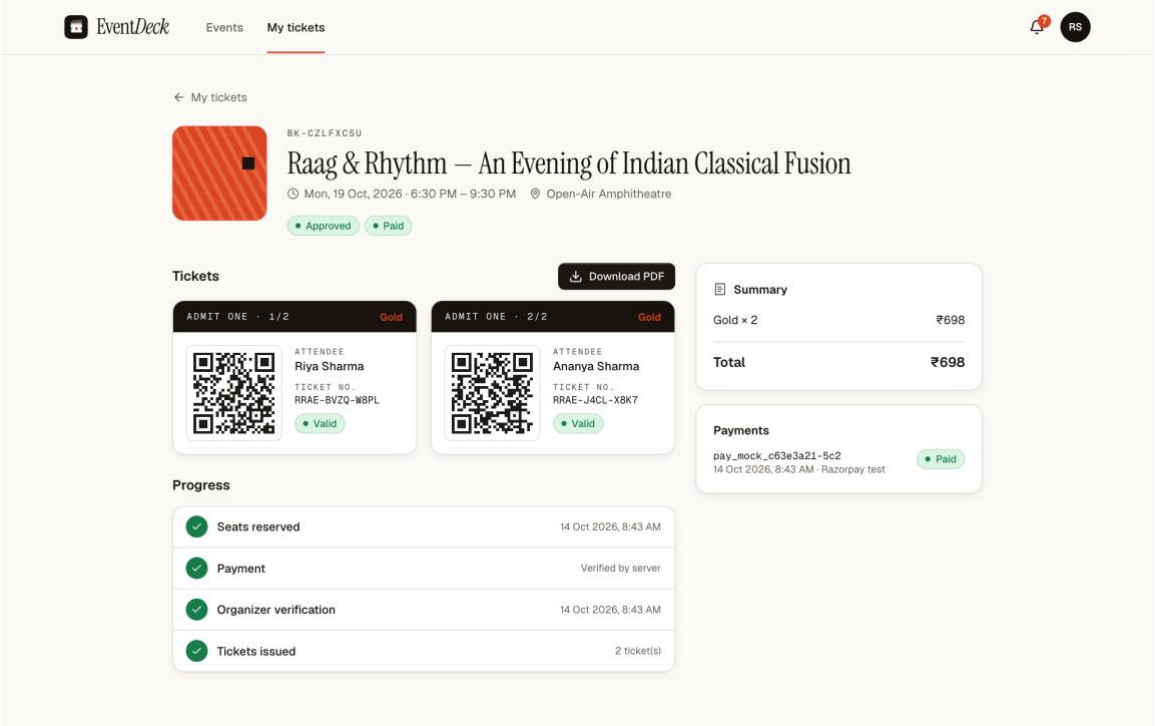


Figure 6.3 Booking Detail with QR Tickets and Progress

6.6 Check-in at the Venue

Figure 6.4 shows how a scan is processed by checkInTicket in src/lib/server/checkin.ts. The scanner page (Figure 6.5) reads QR codes with the phone's camera and also accepts a typed ticket number when a code cannot be read. The server first confirms that the scanning account may scan for this event, then parses the scan, looks up the ticket and checks every condition from the database: the ticket belongs to this event, the event is not cancelled, the ticket is valid, the booking is approved, payment is paid or not required, and the current time is inside the check-in window.

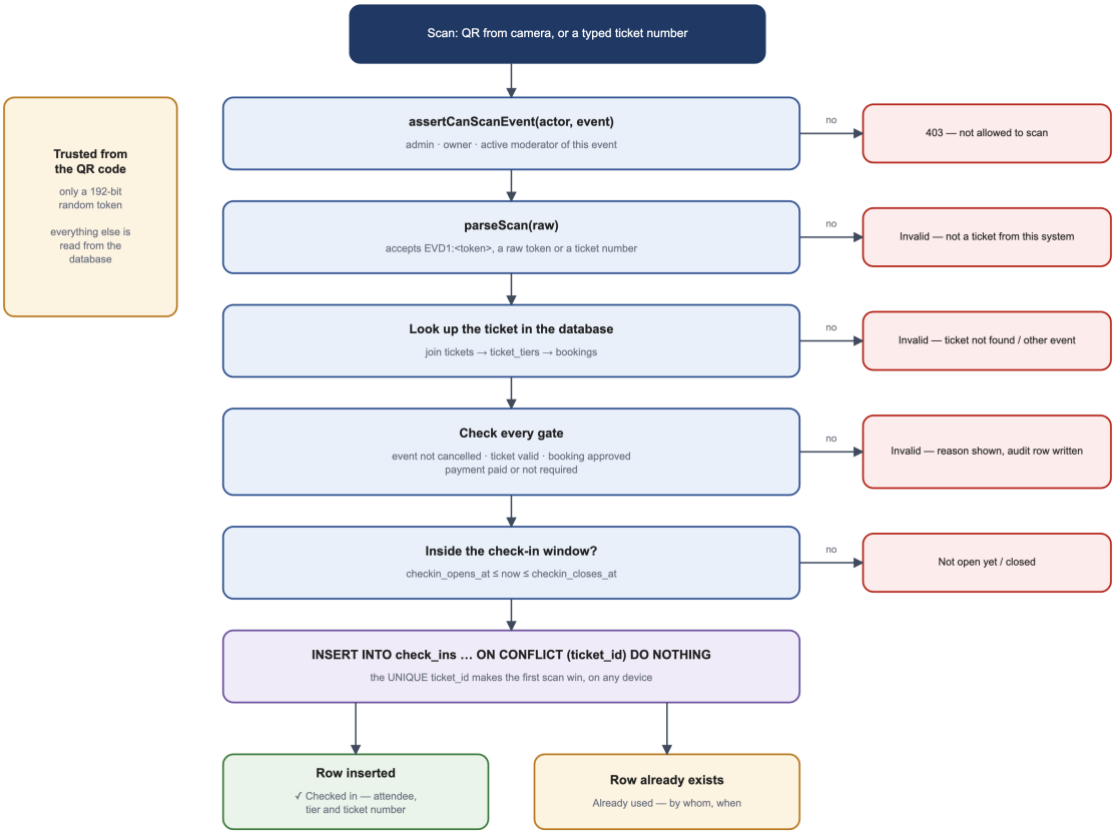


Figure 6.4 Server-Side Check-in Validation

The check-in itself is a single statement, INSERT INTO check_ins … ON CONFLICT (ticket_id) DO NOTHING. Because ticket_id is unique, the database lets exactly one insert succeed for each ticket, however many devices scan it at the same time. The successful scan shows the attendee's name, tier and ticket number; any later scan shows “Already checked in” with the time and the name of the moderator who admitted the ticket. Refused and duplicate scans are written to the audit log.

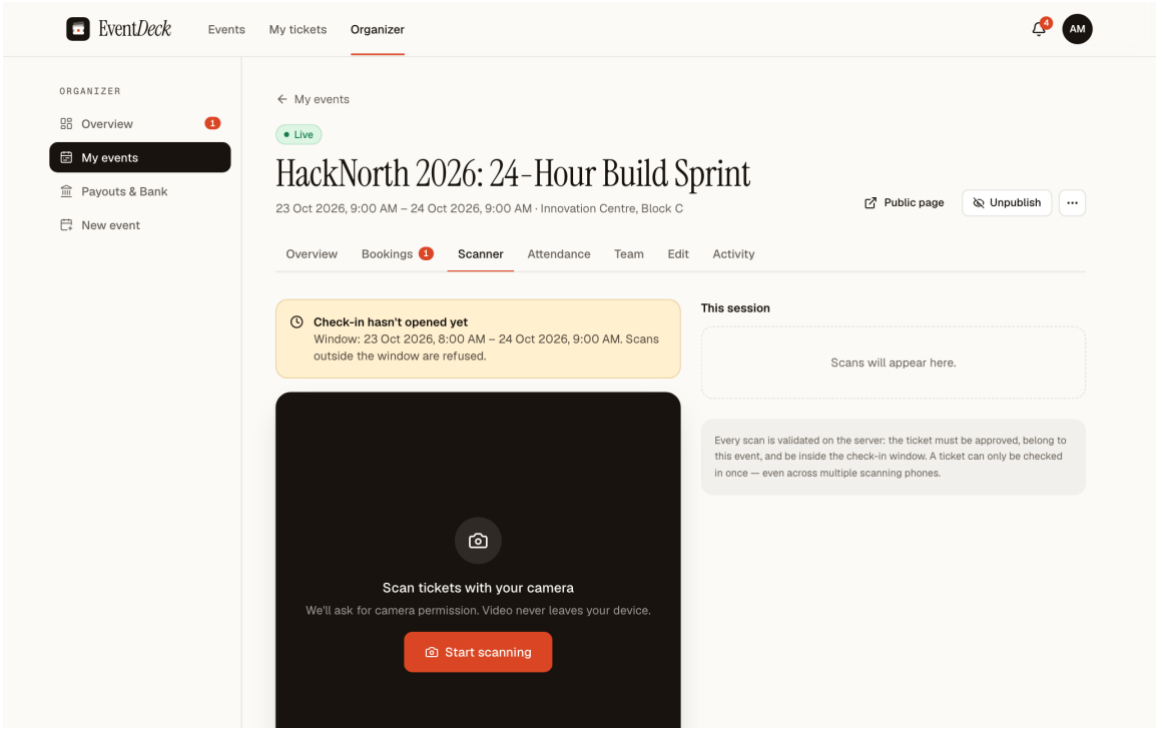


Figure 6.5 Ticket Scanner Page

6.7 Database Constraints as a Safety Net

The application checks every rule before writing, but the most important rules are also declared in the database, so that a mistake in the application code or a request that bypasses it cannot store invalid data. Table 6.3 lists the principal constraints.

Table 6.3 Principal Integrity Constraints

Constraint	Rule enforced
ticket_tiers_allocation	$0 \leq$ allocated $\leq$ capacity for every tier.
events_max_per_booking	The per-person limit is between 1 and 4.
events_time_order, events_registration_before_end, events_checkin_order	End after start; registration closes before the end; check-in closes after it opens.
bookings_idempotency_key (unique)	One booking per participant and idempotency key.
bookings_attendees_match	The number of attendee names equals the quantity.
check_ins.ticket_id (unique)	A ticket can be checked in only once.

Constraint	Rule enforced
tickets.qr_token, tickets.ticket_number (unique)	Every QR token and ticket number is unique.
payments.order_id, payments.payment_id (unique)	An order or payment is recorded once.
webhook_events.event_id (unique)	A webhook delivery is processed once.
refunds.booking_id (unique)	At most one refund per booking.
event_moderators_one_active, cancellation_requests_one_pending	One active assignment per moderator and event; one pending cancellation request per event.
Row-level security	Enabled on all 17 tables; users can read only rows that belong to them, and all writes go through the server.

6.8 Audit Trail and Notifications

Significant actions are recorded in the audit_logs table with the actor, the action, the affected record, the event and a small set of details. Examples are booking.created, booking.approved, booking.expired, payment.captured, checkin.accepted, checkin.rejected, event.published and event.cancellation_requested. Payment secrets and card data are never written to the log. Organizers see the log for their own events on the Activity tab (Figure 6.6), and administrators see the whole log with filters.

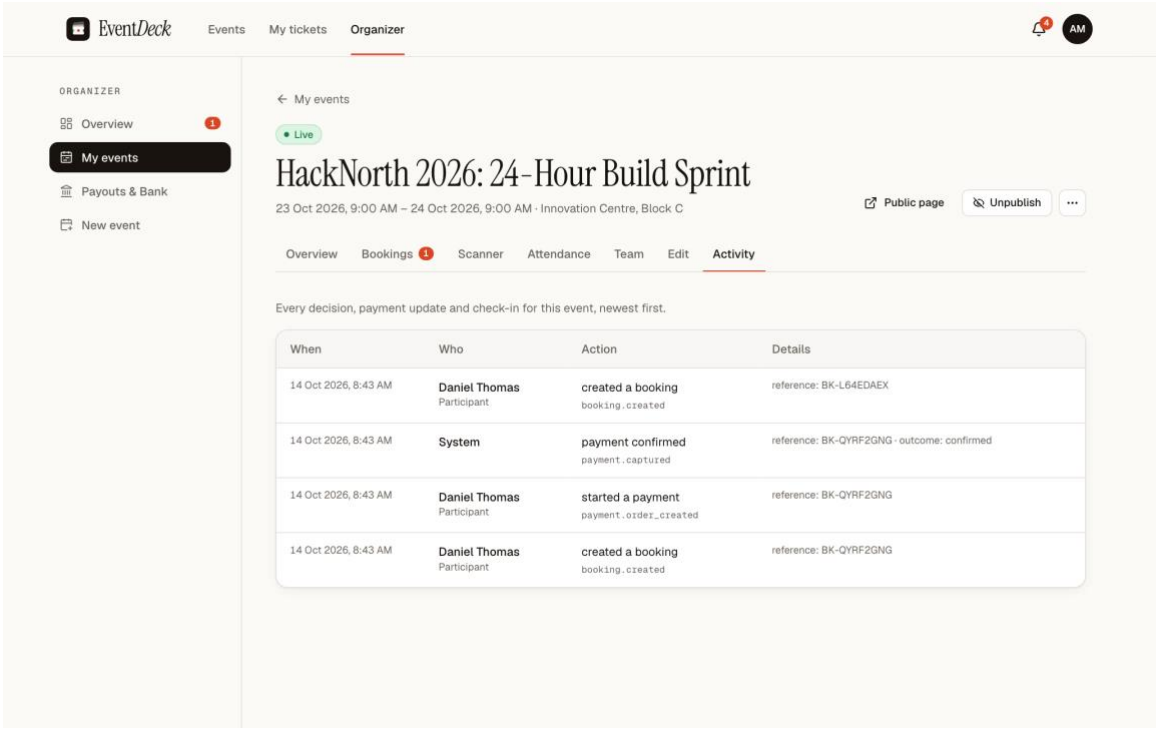


Figure 6.6 Event Activity Log

The same actions create notifications for the people concerned: participants are told when tickets are ready, a hold has expired, a pass request has been decided or an event has been cancelled; organizers are told about new bookings and pass requests; and administrators are told about applications, cancellation requests and payment exceptions. The notification bell updates in real time.

7. SYSTEM IMPLEMENTATION

7.1 Development Environment

The system was developed and tested in the environment summarised in Table 7.1.

Table 7.1 Development and Test Environment

Item	Details
Computer	Apple M4, macOS 26.6.2
Runtime	Node.js 24.16.0, npm 11.13.0
Language	TypeScript 5 (strict mode)
Database	Supabase hosted PostgreSQL (single database platform for development, testing and production)
Editor and tools	Visual Studio Code, drizzle-kit, ESLint 9, Git and GitHub
Browsers	Chromium (Playwright), Safari and Chrome for manual checks; phone and tablet viewports
Payments	Razorpay Test Mode keys; a local payment simulator for development that is disabled in production

7.2 Project Structure

The repository follows the conventions of the Next.js App Router. The source code consists of 166 TypeScript files with about 16,800 lines. The principal directories are described in Table 7.2.

Table 7.2 Project Directory Structure

Path	Contents
src/app/(site)	Public pages: home, event list, event detail, forbidden page.
src/app/(auth)	Sign-in, sign-up, forgot-password and reset-password pages.
src/app/my	Participant area: bookings, booking detail, notifications, profile.
src/app/organizer	Organizer dashboard, events and their tabs, new event, payouts.
src/app/moderate	Moderator area: assigned events and their scanner.

Path	Contents
src/app/admin	Administrator pages: overview, events, people, cancellations, refunds, payouts, exceptions, reports, audit.
src/app/actions, src/app/api	Server Actions and route handlers (Table 4.3).
src/lib/domain	Pure rules: eligibility, pricing, transitions, tokens.
src/lib/server	Database services: bookings, payments, check-in, moderators, events, cancellations, payouts, PDF, storage, audit, queries.
src/db/schema.ts, drizzle/	Schema definition and SQL migrations.
src/components	User-interface components grouped by area.
tests/unit, tests/integration, e2e	Vitest unit and integration tests and Playwright browser tests.
scripts	Administrator creation, demonstration data and poster seeding.

7.3 Database Schema and Migrations

The schema is declared in TypeScript with Drizzle in src/db/schema.ts, from which drizzle-kit generates SQL migrations. Security features that Drizzle cannot express — row-level security policies, the profile-provisioning trigger, the realtime publication and the storage buckets — are written by hand in SQL. Migrations are applied in order with drizzle-kit migrate, which records each applied file. All eight migrations listed in Table 7.3 are applied to the hosted project.

Table 7.3 Database Migrations

File	Change
0000_init.sql	Enumerations and core tables: profiles, events, tiers, bookings, items, tickets, check-ins, payments, refunds, notifications, audit.
0001_security_rls_realtime.sql	Profile provisioning trigger, role helper functions, row-level security policies, realtime publication, storage buckets.
0002_institution_onboarding.sql	Institution type, campus and affiliation on profiles.
0003_srm_email_gate.sql	Signup trigger accepts SRM student classification only for SRM e-mail domains (fix QA-001).
0004_moderators_and_audience.sql	event_moderators table and audience columns on events.
0005_moderators_rls.sql	Row-level security for moderator assignments.

File	Change
0006_odd_maelstrom.sql	Organizer payout accounts, payouts and cancellation-request support.
0007_max_four_tickets_per_person.sql	Lowers existing limits above four and changes the CHECK constraint to between 1 and 4.

7.4 Authentication Implementation

Sign-in, sign-up and password management are implemented in src/app/actions/auth.ts using the server-side Supabase client from src/lib/supabase/server.ts. After sign-in the user is sent to the page appropriate to their role: participants to the event list, organizers to their dashboard and administrators to the administration overview, unless a next parameter names another page. Administrator accounts are created only with npm run admin:create, which reads the address and password from environment variables or generates a strong password. Table 7.4 summarises the parameters.

Table 7.4 Authentication Parameters

Parameter	Value
Identity provider	Supabase Auth, e-mail and password
Password rule	8–72 characters with at least one letter and one number
Session storage	HTTP-only cookies managed by @supabase/ssr; refreshed by the edge proxy
Session check	supabase.auth.getUser() on every server request, memoised per request
Deactivated accounts	Treated as signed out on the next request
Role on sign-up	Always participant; role in sign-up data is ignored
Password reset	One-time token by e-mail exchanged at /auth/confirm; no account enumeration
Administrator creation	Command-line script only

7.5 Participant Interface

After signing in, a participant lands on the event list. Booking an event takes three steps on the event page: choose tickets in each tier, enter the attendee name for each ticket and confirm. Free bookings show the tickets immediately; paid bookings open Razorpay Checkout and show a ten-minute countdown. The My tickets page (Figure 7.1) separates upcoming, past and all bookings and shows each booking's status and amount. Notifications are listed in an inbox reached from the bell icon.

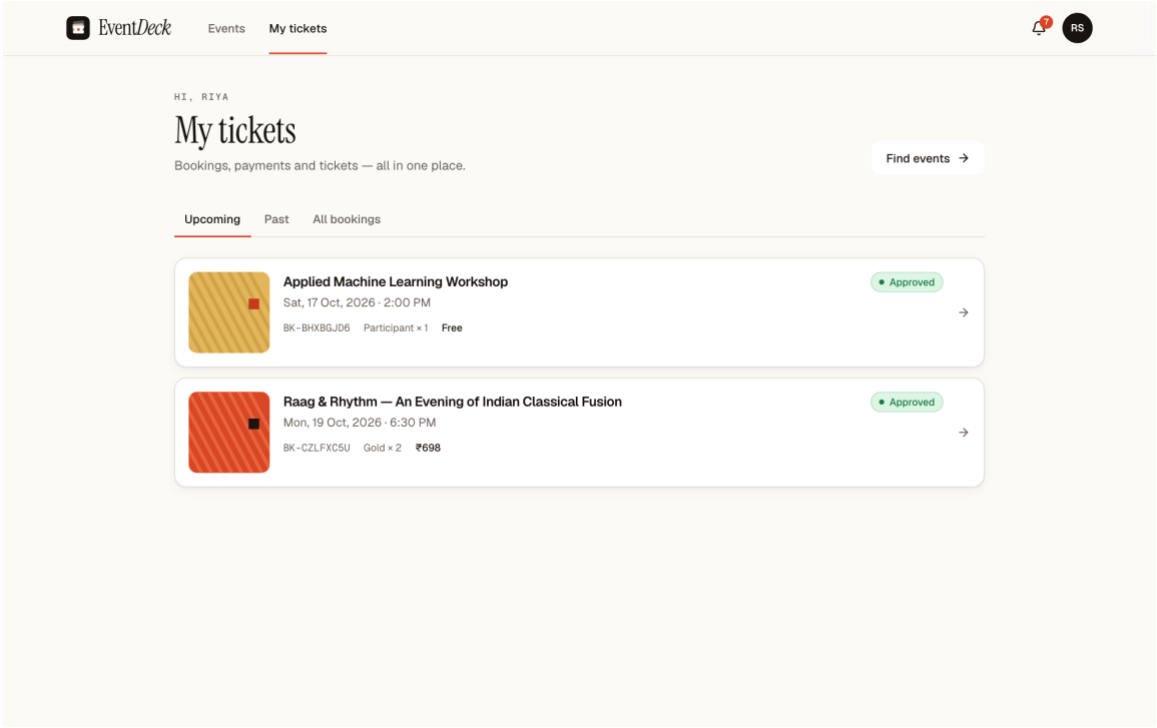


Figure 7.1 My Tickets Page of a Participant

7.6 Organizer Interface

The organizer dashboard (Figure 7.2) shows upcoming events, approved tickets, test-mode revenue, the verification queue and a table of the organizer's events with seats, pending items, revenue and check-ins. Each event has tabs for Overview, Bookings, Scanner, Attendance, Team, Edit and Activity. The Overview (Figure 7.3) shows seats taken, approved tickets, items to verify, bookings awaiting payment, revenue, inventory by tier and the check-in window.

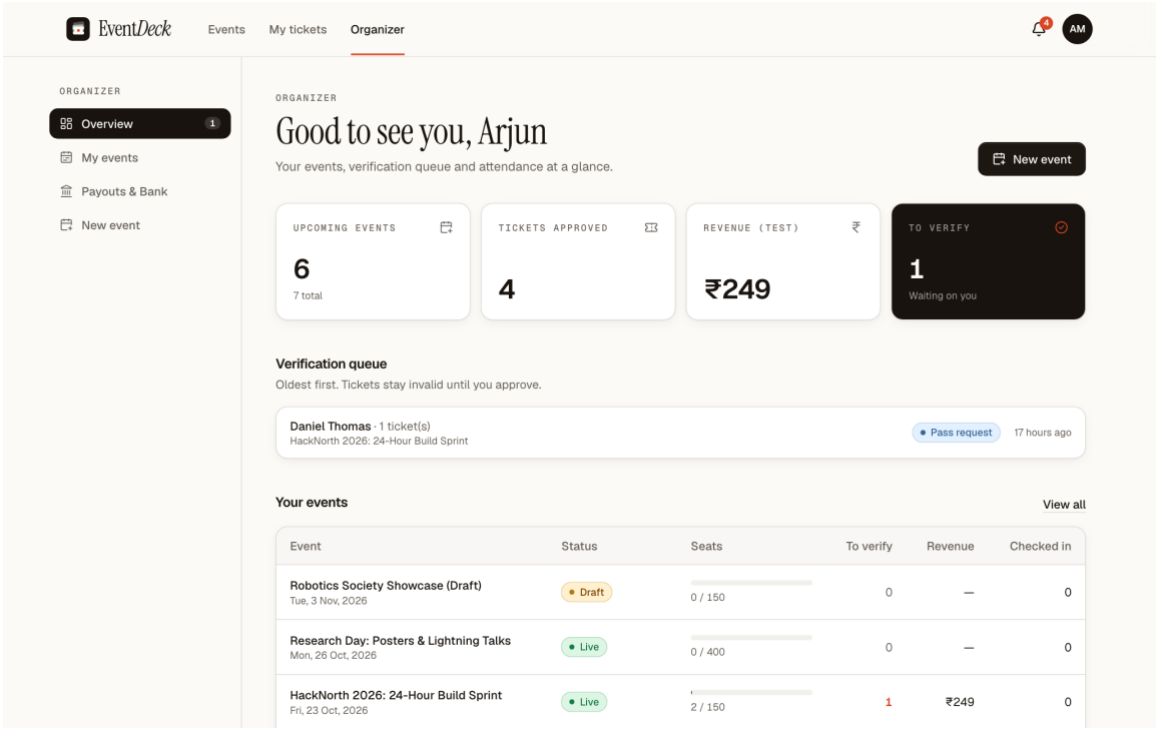


Figure 7.2 Organizer Dashboard

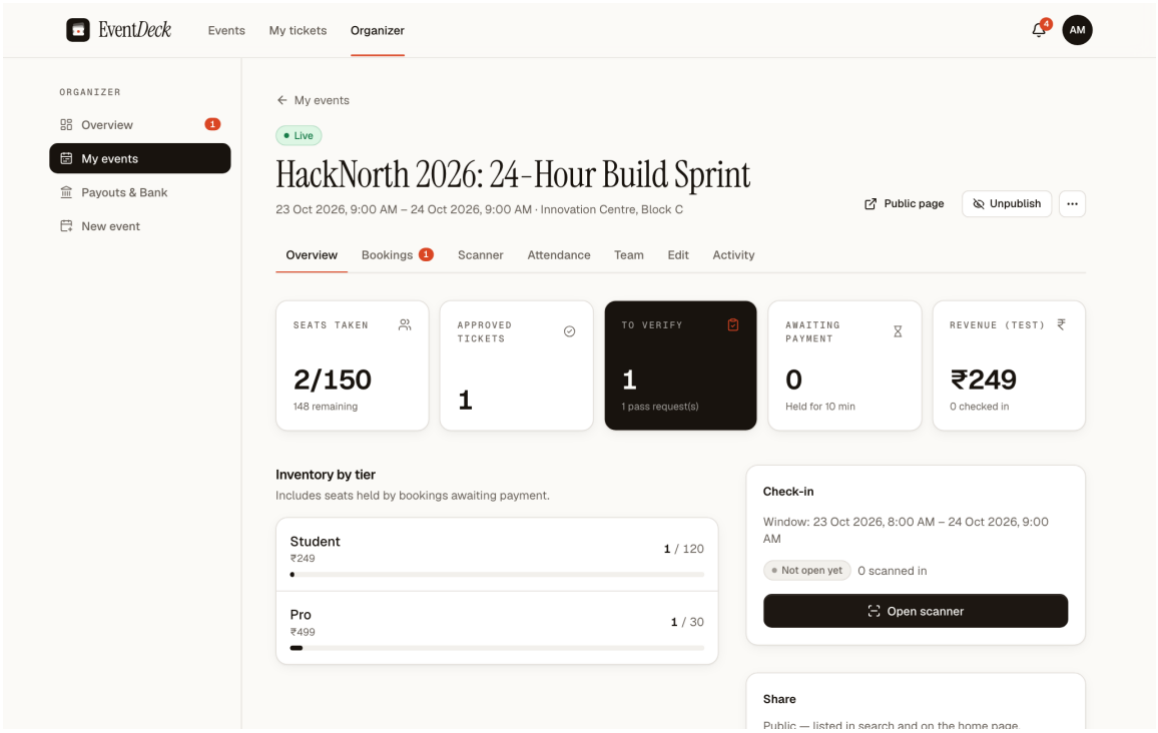


Figure 7.3 Event Overview with Inventory by Tier

The Attendance tab (Figure 7.4) lists every valid ticket with the attendee, tier, ticket number, the person who booked it and whether it has been checked in and by whom, and it can be exported as a CSV file.

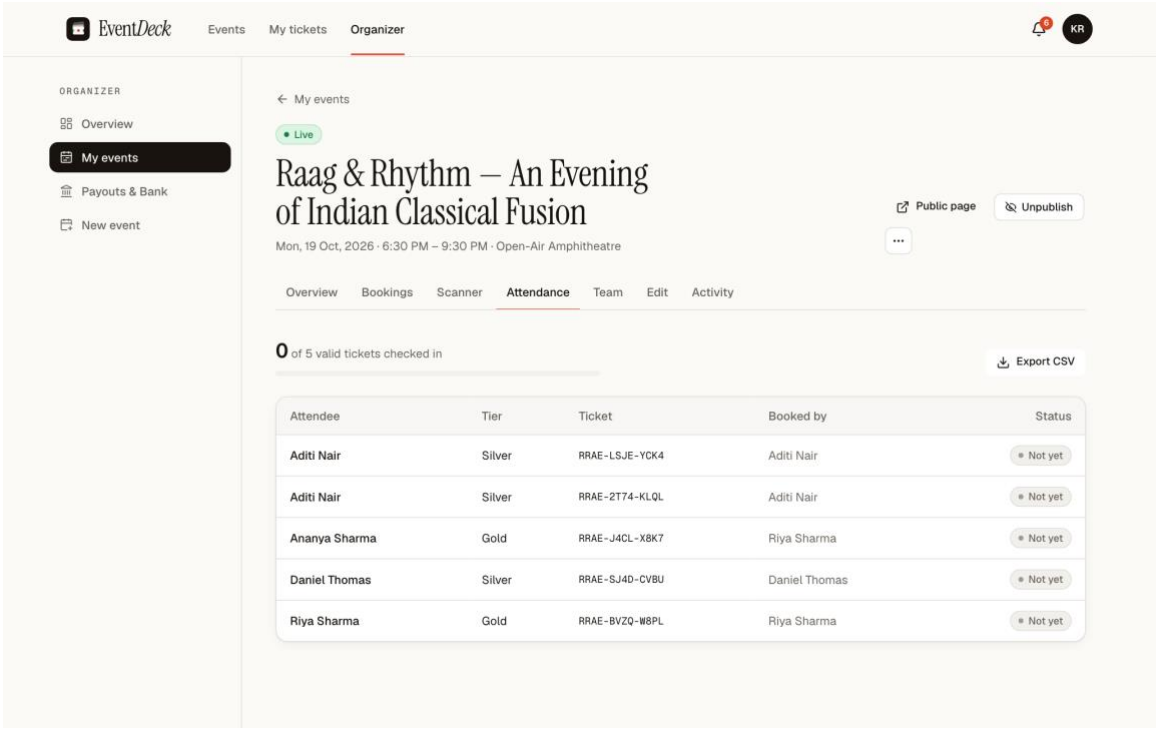


Figure 7.4 Attendance Page for an Event

7.7 Administration, Cancellations and Refunds

The administration overview (Figure 7.5) summarises live events, approved tickets, test-mode payments and check-ins, shows ticket sales for the last fourteen days and lists the items that need attention: organizer applications, cancellation requests and payment exceptions. The sidebar badges show how many items are waiting in each queue.

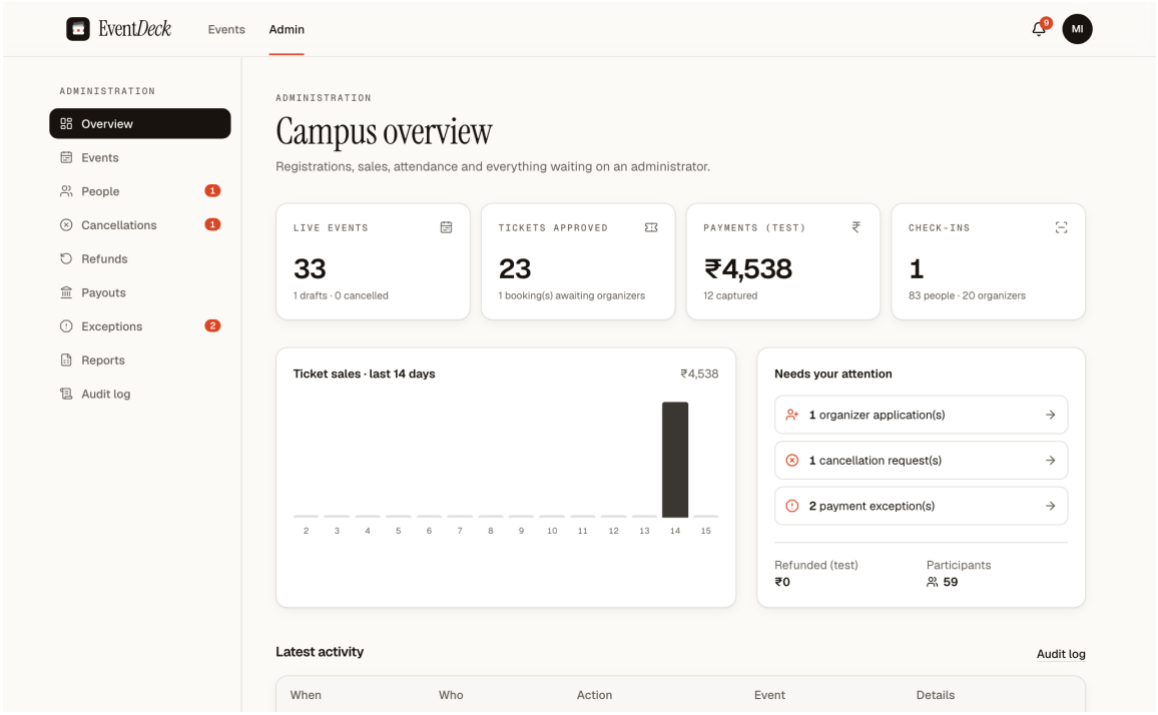


Figure 7.5 Administrator Campus Overview

Cancellation requests (Figure 7.6) show the event, the reason and the number of live and paid bookings that would be affected. Approving a request runs the cancellation transaction described in Section 5.4. Refunds are then listed on the Refunds page, where an administrator processes them through Razorpay Test Mode; the refund status is also updated by Razorpay's refund webhooks. The Exceptions page lists payments that could not be applied normally, which an administrator marks as resolved with a note after handling them.

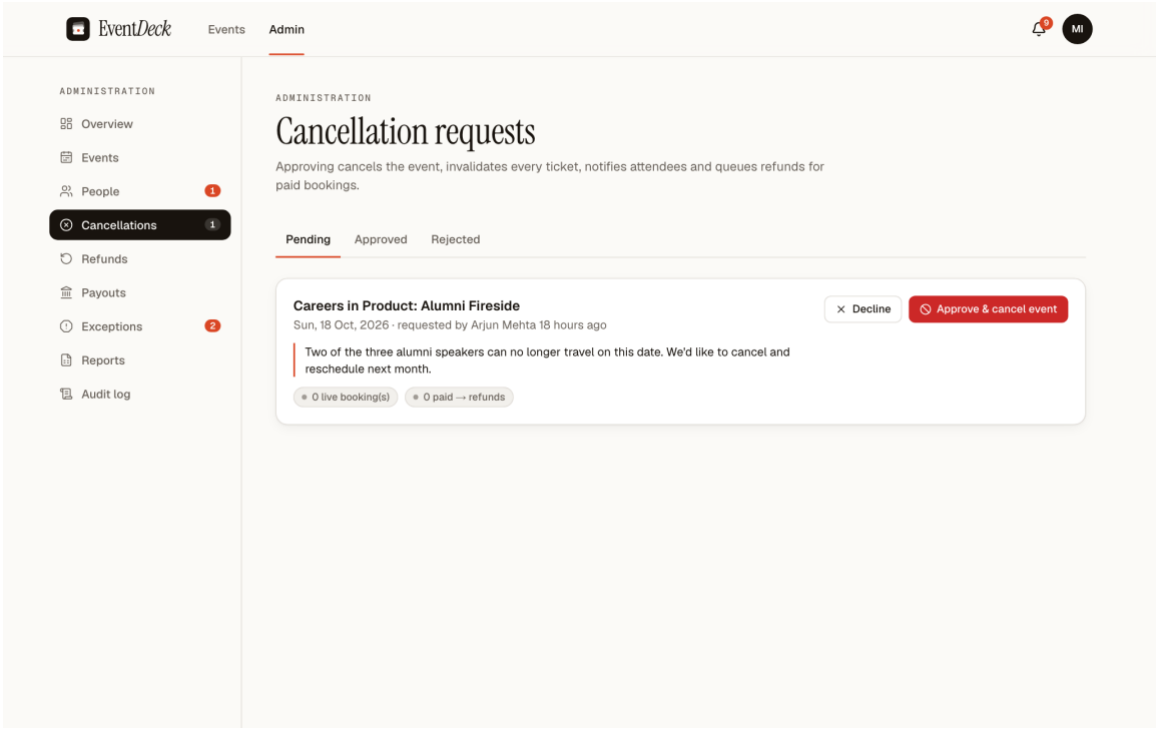


Figure 7.6 Cancellation Requests Awaiting Review

7.8 Reports, Payouts and Exports

The Reports page (Figure 7.7) offers CSV exports of registrations, payments, events and sales, refunds, people and the latest audit entries. Exports are generated on the server from live data; every exported text field that begins with =, +, - or @ is prefixed so that a spreadsheet will not treat it as a formula, and each export is itself recorded in the audit log.

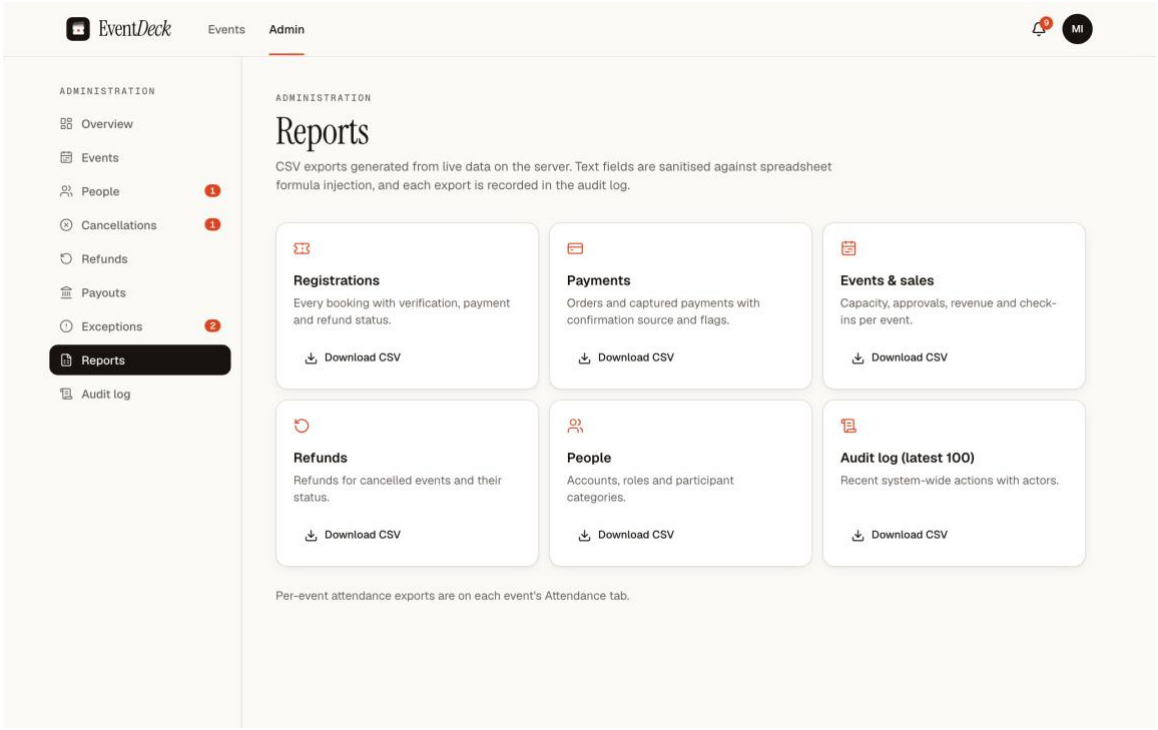


Figure 7.7 Reports Page with CSV Exports

Organizers who sell paid tickets register a bank account (with IFSC code) or a UPI identifier and request a payout for an event. The amount requested may not exceed the paid ticket revenue of the event minus earlier requests, and a platform fee of five per cent is deducted. An administrator completes the payout by entering the bank transfer reference (UTR) or rejects it with a note. The system records payouts but does not move money; the transfer is made through the university's normal banking process.

7.9 Configuration and Deployment Considerations

The application is configured through environment variables, listed in Table 7.5. Their values are kept in .env.local for development and in the hosting provider's settings for production; the repository contains only .env.example files with empty values, and no secret is reproduced in this report.

Table 7.5 Configuration Variables

Variable	Purpose
NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY	Supabase project address and public key used by browsers

Variable	Purpose
SUPABASE_SECRET_KEY	Server-only key for storage and administrative calls
DATABASE_URL, DIRECT_URL	Pooled connection used by the application and direct connection used for migrations
NEXT_PUBLIC_APP_URL	Public address used in e-mail links and redirects
NEXT_PUBLIC_UNIVERSITY_NAME	Name shown in the interface
RAZORPAY_KEY_ID, RAZORPAY_KEY_SECRET, NEXT_PUBLIC_RAZORPAY_KEY_ID	Razorpay Test Mode keys; live keys are refused
RAZORPAY_WEBHOOK_SECRET	Secret used to verify webhook signatures
ALLOW_MOCK_PAYMENTS	Enables the local payment simulator outside production
CRON_SECRET	Protects the scheduled hold-expiry route
ADMIN_EMAIL, ADMIN_NAME, ADMIN_PASSWORD	Used once by the administrator-creation script
SEED_DEMO_PASSWORD	Password for the fictional demonstration accounts

The project includes a Vercel configuration that runs the hold-expiry job once a day as a safety net; holds are also released lazily on every booking. The Supabase project must have the eight migrations applied, Realtime enabled for the bookings and notifications tables, and the Razorpay webhook URL registered with the webhook secret.

8. TESTING AND RESULTS

8.1 Testing Methodology

Testing combined three activities. The first is a suite of automated tests at three levels: unit tests of the pure rules, which need no database; integration tests that call the real service functions against a PostgreSQL database with authentication and storage; and end-to-end tests that drive the application in a browser as participants, organizers, moderators and administrators. The second is static checking with the TypeScript compiler and ESLint. The third is a walkthrough of every page with fictional demonstration data, during which the screenshots in this report were captured.

The integration and end-to-end tests deliberately include adversarial and concurrent cases: many buyers competing for the last seats, two moderators scanning the same ticket at once, a payment confirmed by the browser and two webhooks at the same time, forged QR codes, requests with another user's identifiers and direct calls to the database API with a user's token. To protect real data, the test configuration refuses to run unless it points at a dedicated Supabase test project different from the main project.

8.2 Test Environment

The test run reported in this chapter was carried out in the environment shown in Table 8.1. For this report the unit tests, the type check and the linter were run again on the current code. The integration and end-to-end suites could not be run again, because the dedicated Supabase test project they require has not yet been created; their status is reported separately and honestly below.

Table 8.1 Test Environment for the Reported Run

Item	Details
Date and time	15 October 2026, 02:06 IST
Code version	Commit e9a4c24 with the uncommitted working-tree changes current on that date
Machine	Apple M4, macOS 26.6.2
Runtime	Node.js 24.16.0, npm 11.13.0
Test runner	Vitest 5.0 (unit project); Playwright 1.64 configured but not run
Database	None for unit tests; walkthrough on the hosted Supabase project with fictional data

Item	Details
Command	npx vitest run --project unit

8.3 Test Suites and Overall Results

Table 8.2 lists the automated test files, the number of tests in each and the outcome in the reported run. All 59 unit tests in six files passed in 988 milliseconds. The 53 integration tests and 24 end-to-end tests were not executed in this run. During the project's quality-assurance phase on 14 October 2026 the then-current suites passed in full — 94 of 94 Vitest tests (41 unit and 53 integration) and 26 of 26 Playwright runs — against a local Supabase stack that has since been removed so that Supabase's hosted PostgreSQL is the only database platform. Those earlier results are cited in Sections 8.5 and 8.6 as historical evidence and are labelled as such.

Table 8.2 Automated Test Suites and Results

No.	Suite	File	Tests	Result
1	Unit	tests/unit/eligibility.test.ts	9	9 passed, 0 failed
2	Unit	tests/unit/domain.test.ts	15	15 passed, 0 failed
3	Unit	tests/unit/auth-onboarding.test.ts	14	14 passed, 0 failed
4	Unit	tests/unit/payouts.test.ts	10	10 passed, 0 failed
5	Unit	tests/unit/security.test.ts	4	4 passed, 0 failed
6	Unit	tests/unit/url.test.ts	7	7 passed, 0 failed
7	Integration	tests/integration/api-security.test.ts	4	Not run (no test project)
8	Integration	tests/integration/auth-authorization.test.ts	14	Not run (no test project)
9	Integration	tests/integration/booking-engine.test.ts	17	Not run (no test project)
10	Integration	tests/integration/eligibility-bypass.test.ts	5	Not run (no test project)
11	Integration	tests/integration/moderators.test.ts	10	Not run (no test project)
12	Integration	tests/integration/storage.test.ts	2	Not run (no test project)
13	Integration	tests/integration/webhook-route.test.ts	1	Not run (no test project)
14	End-to-end	e2e/*.spec.ts (11 files)	24	Not run (no test project)
	Total executed		59	59 passed, 0 failed

The complete list of unit test names and their results is given in Appendix 11.3. The following sections describe the main groups of tests.

8.4 Functional Test Cases

Table 8.3 lists representative functional test cases from the unit suite executed for this report. The test names are those used in the code.

Table 8.3 Representative Functional Test Cases

ID	Test case	Expected behaviour	Result
TC-01	booking state machine › allows only the documented transitions	Only moves listed in BOOKING_TRANSITIONS are accepted	Passed
TC-02	booking state machine › tickets are valid only when every gate passes	Valid only if approved, paid or free, and event not cancelled	Passed
TC-03	server-side pricing › prices from database rows and merges duplicate lines	Amount computed from tier rows, never from input	Passed
TC-04	server-side pricing › rejects empty, oversized, negative, fractional and unknown lines	Invalid orders refused	Passed
TC-05	platform limit: 4 tickets per person › event validation refuses a per-person limit above 4	Limit of 5 or more rejected	Passed
TC-06	event audience eligibility › campus-only events accept only that campus	Other campuses and colleges refused	Passed
TC-07	event audience eligibility › inter-college events accept only approved institutions	Names matched after normalisation	Passed
TC-08	event audience eligibility › booking requests cannot smuggle profile fields past validation	Extra fields stripped	Passed
TC-09	tickets & QR › generates unguessable, unique tokens	192-bit tokens, no repeats	Passed
TC-10	tickets & QR › parses QR payloads, raw tokens and typed ticket numbers; rejects junk	Only well-formed input accepted	Passed

ID	Test case	Expected behaviour	Result
TC-11	CSV export safety › neutralises spreadsheet formulas and quotes properly	Formula prefixes escaped	Passed
TC-12	validation schemas › signup never accepts a role and requires a reasonable password	Role ignored; weak passwords refused	Passed
TC-13	validation schemas › enforces SRM organization email for host students	SRM students need an SRM address	Passed
TC-14	payout validation schemas › fee calculations › computes 5% platform fee correctly	Fee and net amount correct	Passed

8.5 Concurrency and Data Integrity Tests

The integration suite contains tests that start many operations at the same moment and then inspect the database. Table 8.4 lists them with the result recorded in the quality-assurance run of 14 October 2026. They were not re-run for this report for the reason given in Section 8.2.

Table 8.4 Concurrency and Integrity Tests (Historical Results)

ID	Test	Observed result	Status
CI-01	20 concurrent bookings for a tier with 5 seats	Exactly 5 succeeded; allocated = 5	Passed 14 Oct; not re-run
CI-02	Same booking submitted twice with one idempotency key	One booking created	Passed 14 Oct; not re-run
CI-03	4 concurrent scans of one ticket by two moderators	Exactly 1 check-in row	Passed 14 Oct; not re-run
CI-04	Checkout callback and 2 webhook deliveries racing	Exactly 1 confirmation	Passed 14 Oct; not re-run
CI-05	3 concurrent hold-expiry sweeps	Seats released exactly once	Passed 14 Oct; not re-run
CI-06	Late payment when seats are free / gone	Recovered / payment exception, never a ticket	Passed 14 Oct; not re-run
CI-07	Event cancellation with paid bookings	Tickets invalidated, refunds queued	Passed 14 Oct; not re-run

To obtain current evidence about seat accounting, a read-only query was run on the hosted demonstration database on 15 October 2026. For every tier it compared the allocated count with the seats held by live bookings (awaiting payment, pending verification or approved). Table 8.6 in the next section shows the result for the demonstration events; all 39 tiers in the database, including those of leftover test events, were consistent.

8.6 Access Control and Isolation Tests

Access-control tests act as different users and attempt operations that should be refused, including requests that substitute another user's identifiers and direct calls to the Supabase database API. Table 8.5 summarises them; the results marked as historical come from the run of 14 October 2026.

Table 8.5 Access Control and Isolation Tests

ID	Test	Result
AC-01	Participant opens organizer or admin pages → /forbidden; anonymous → /login	Passed (historical)
AC-02	Another organizer opens an event they do not own → not found; CSV → 403	Passed (historical)
AC-03	Moderator of another event, revoked or deactivated moderator scans → refused	Passed (historical)
AC-04	Participant downloads another participant's ticket PDF → not found	Passed (historical)
AC-05	Create payment order for another user's booking → refused	Passed (historical)
AC-06	Database API with a user token reads others' bookings, payments, tickets, check-ins → empty	Passed (historical)
AC-07	Database API writes: self-promotion to admin, forged moderator, approving a booking → rejected	Passed (historical)
AC-08	Direct Auth sign-up claiming SRM student with a non-SRM e-mail → classified as guest	Passed (historical)
AC-09	Webhook with a bad signature → 401, no change	Passed (historical)
AC-10	Razorpay signature verification accepts correct and rejects tampered signatures	Passed (this run)

ID	Test	Result
AC-11	Live payment keys refused; simulator disabled in production	Passed (this run)
AC-12	Poster upload type decided from file bytes, not the file name	Passed (this run)
AC-13	Three roles map to the correct dashboards	Passed (this run)

8.7 Static Checks and Interface Walkthrough

The TypeScript compiler (tsc --noEmit) completed with exit code 0 and no errors. ESLint completed with 0 errors and 21 warnings; the warnings are informational notes from the React Compiler about React Hook Form's watch() function and do not indicate incorrect behaviour. A production build was last run during the quality-assurance phase on 14 October 2026, when it succeeded; it was not repeated for this report.

For the walkthrough, every participant, organizer and administrator page was opened with the fictional demonstration accounts and captured in a headless Chromium browser at 1366 × 860 pixels. The walkthrough was not a formal test with pass criteria for every page, but it produced the evidence in Table 8.6 and the observations in Table 8.7.

Table 8.6 Seat Inventory Consistency Check on the Demonstration Database

Event	Tier	Capacity	Allocated	Seats in live bookings	Consistent
Orientation Mixer for New Students	Entry	250	2	2	Yes
Midnight Film Club: Restored Classics	Screening	6	3	3	Yes
Applied Machine Learning Workshop	Participant	40	1	1	Yes
Raag & Rhythm	Silver	200	3	3	Yes
Raag & Rhythm	Gold	80	2	2	Yes
Raag & Rhythm	Diamond	12	0	0	Yes
Inter-College Football Cup — Finals	Pavilion	60	3	3	Yes

Event	Tier	Capacity	Allocated	Seats in live bookings	Consistent
Inter-College Football Cup — Finals	Stands	600	0	0	Yes
HackNorth 2026	Student	120	1	1	Yes
HackNorth 2026	Pro	30	1	1	Yes
Ten other demonstration tiers	—	—	0	0	Yes
All tiers in the database	39 tiers				39 of 39

Table 8.7 Defects and Observations

ID	Observation	Cause identified	Status
QA-001	Anyone could classify themselves as an SRM student through a direct Auth sign-up	Signup trigger trusted client metadata	Fixed (migration 0003)
QA-002	External users could change their profile to SRM student	Profile update did not re-check the e-mail domain	Fixed
QA-003, QA-004	Organizers could not delegate scanning; events could not be limited by campus or college	Missing features	Implemented
QA-005, QA-009	Tests were wired to the hosted database, then to a local Docker stack	Test configuration	Fixed: dedicated test project with guards
QA-006 – QA-008	Broken signup test helper, schema snapshot drift, lint scanning build output	Tooling	Fixed
OBS-1	Administrator pages hang or fail intermittently on the hosted database	Many parallel queries through the transaction pooler (port 6543); the same pages load in about 1.5 s through the session pooler	Open

ID	Observation	Cause identified	Status
OBS-2	Development server log prints sign-in form values, including passwords	Development-mode logging of Server Action arguments	Open (development only)
OBS-3	Leftover integration-test events are visible in the public event list of the hosted project	Test data created before the test-project guard existed	Open; clean-up awaiting approval
OBS-4	Attendance table has no pagination for very large events	Not implemented	Open (low priority)

Observation OBS-1 was found while capturing the screenshots for this report: every administrator page waited for more than ninety seconds when the application connected through Supabase's transaction-mode pooler, while organizer and participant pages were unaffected. Running the same development server with the session-mode connection string made every administrator page load in about 1.2 to 3.2 seconds, which identifies the connection mode as the cause. The administrator screenshots were captured in that configuration. A permanent fix — reducing the number of parallel queries in the administrator layout, or changing the connection settings — is listed as future work.

8.8 Limitations of the Testing Approach

The evidence presented in this chapter shows that the implemented rules behave as intended in the cases tested, but it has limits. The most important is that the integration and end-to-end suites, which contain the concurrency and access-control tests, were last executed on 14 October 2026 against a local stack that no longer exists, and must be re-run on a dedicated Supabase test project before the results can be called current. Table 8.8 lists the testing that was not executed.

Table 8.8 Testing Not Executed

Type of testing	Status	Remarks
Integration tests (53)	Not executed in this run	Need a dedicated Supabase test project and .env.test; passed on 14 Oct on the earlier stack

Type of testing	Status	Remarks
End-to-end tests (24 tests, 26 runs)	Not executed in this run	Same requirement; passed on 14 Oct on the earlier stack
Load and performance testing	Not executed	Concurrency tests check correctness, not capacity
Physical phone camera over HTTPS	Not executed	Needs a deployed HTTPS address; camera decoding was tested with a simulated webcam
Razorpay Checkout with a test card	Not executed	Third-party payment window; server confirmation is covered by tests
E-mail delivery on the hosted project	Not executed	Supabase's free e-mail service is rate-limited
Formal accessibility audit	Not executed	Keyboard navigation and phone layouts were checked by end-to-end tests
User acceptance testing	Not executed	No pilot with real organizers yet

Within these limits, the results support the main claims of the design: seat accounting was consistent on every tier of the demonstration database, the domain rules that decide prices, limits, eligibility, token formats and signatures pass all of their tests, and the concurrency and access-control tests passed when they were last executed.

9. CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

This project set out to build an event ticketing system for a multi-campus university that never oversells an event, applies audience rules from trusted data, confirms payments with the payment provider and admits each ticket exactly once. EventDeck meets these aims by keeping every seat change in a database transaction with a conditional update, by evaluating eligibility and limits on the server from stored profiles, by confirming Razorpay payments only after signature verification, and by letting a unique constraint decide which scan of a ticket is the first.

Around this core, the system provides the tools a university needs to run events: a four-step sign-up that models SRM campuses and other colleges, organizer applications, an event form with tiers, windows and audience rules, a poster-led discovery experience, event-specific moderators for the door, cancellation and refund handling, payouts, notifications, an audit log and CSV reports.

The work also showed the value of testing against a real database and of walking through the interface. Quality assurance found two serious defects in which users could claim SRM membership they did not have, and both were fixed with regression tests. Preparing this report revealed a connection-pooling problem on the administrator pages that unit tests could not have found. These experiences confirm that the rules of an event system belong in the database and on the server, and that they must be tested there.

9.2 Future Enhancements

1. Resolve the recorded observations – reduce parallel queries on the administrator pages or adjust the pooler connection, stop development logs from printing form values, and remove the leftover test events from the hosted project.
2. Re-run the full test suite – create the dedicated Supabase test project and run the integration and end-to-end suites on every change.
3. Waiting lists – let participants join a queue for sold-out tiers and offer released seats in order.
4. Invite-only events – restrict an event to a list of named people in addition to the existing audience rules.
5. Live payments and automated payouts – after review, move from Razorpay Test Mode to live keys and use bank payouts through the provider.

6. Load and acceptance testing – measure behaviour with realistic numbers of simultaneous buyers and pilot the system with real organizers.
7. Offline-tolerant scanning – cache the ticket list for a door device so that check-in continues briefly during network outages.
8. Accessibility and mobile review – carry out a formal accessibility audit and test on a wider range of phones and browsers.

10. REFERENCES

[1] J. Gray and A. Reuter, *Transaction Processing: Concepts and Techniques*. San Mateo, CA, USA: Morgan Kaufmann, 1993.

[2] E. F. Codd, "A relational model of data for large shared data banks," *Communications of the ACM*, vol. 13, no. 6, pp. 377–387, 1970.

[3] P. P.-S. Chen, "The entity-relationship model—Toward a unified view of data," *ACM Transactions on Database Systems*, vol. 1, no. 1, pp. 9–36, 1976.

[4] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. New York, NY, USA: McGraw-Hill, 2019.

[5] K. P. Eswaran, J. N. Gray, R. A. Lorie, and I. L. Traiger, "The notions of consistency and predicate locks in a database system," *Communications of the ACM*, vol. 19, no. 11, pp. 624–633, 1976.

[6] H. Berenson, P. Bernstein, J. Gray, J. Melton, E. O'Neil, and P. O'Neil, "A critique of ANSI SQL isolation levels," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 1995, pp. 1–10.

[7] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA, USA: O'Reilly Media, 2017.

[8] The PostgreSQL Global Development Group, "Explicit locking," *PostgreSQL Documentation*, ch. 13.3. [Online]. Available: https://www.postgresql.org/docs/current/explicit-locking.html (accessed Oct. 15, 2026).

[9] The PostgreSQL Global Development Group, "Transaction isolation," *PostgreSQL Documentation*, ch. 13.2. [Online]. Available: https://www.postgresql.org/docs/current/transaction-iso.html (accessed Oct. 15, 2026).

[10] The PostgreSQL Global Development Group, "INSERT," *PostgreSQL Documentation*, SQL Commands. [Online]. Available: https://www.postgresql.org/docs/current/sql-insert.html (accessed Oct. 15, 2026).

[11] J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975.

[12] D. F. Ferraiolo and D. R. Kuhn, “Role-based access controls,” in *Proc. 15th National Computer Security Conf.*, Baltimore, MD, USA, 1992, pp. 554–563.

[13] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, “Role-based access control models,” *IEEE Computer*, vol. 29, no. 2, pp. 38–47, 1996.

[14] Supabase, “Row Level Security,” *Supabase Documentation*. [Online]. Available: https://supabase.com/docs/guides/database/postgres/row-level-security (accessed Oct. 15, 2026).

[15] OWASP Foundation, “OWASP Top 10:2021.” [Online]. Available: https://owasp.org/Top10/ (accessed Oct. 15, 2026).

[16] OWASP Foundation, “Application Security Verification Standard 4.0.3,” 2021. [Online]. Available: https://owasp.org/www-project-application-security-verification-standard/ (accessed Oct. 15, 2026).

[17] P. A. Grassi et al., “Digital identity guidelines: Authentication and lifecycle management,” National Institute of Standards and Technology, Gaithersburg, MD, USA, NIST SP 800-63B, 2017.

[18] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” Internet Engineering Task Force, RFC 7519, 2015.

[19] D. Eastlake 3rd, J. Schiller, and S. Crocker, “Randomness requirements for security,” Internet Engineering Task Force, RFC 4086, 2005.

[20] *Information technology — Automatic identification and data capture techniques — QR Code bar code symbology specification*, ISO/IEC 18004:2015, 2015.

[21] Razorpay, “Payment Gateway” and “Webhooks,” *Razorpay Docs*. [Online]. Available: https://razorpay.com/docs/ (accessed Oct. 15, 2026).

[22] J. Jena and S. Dalal, “The Idempotency-Key HTTP header field,” Internet Engineering Task Force, Internet-Draft draft-ietf-httpapi-idempotency-key-header (work in progress).

[23] OWASP Foundation, “CSV Injection.” [Online]. Available: https://owasp.org/www-community/attacks/CSV_Injection (accessed Oct. 15, 2026).

[24] Vercel, “Next.js documentation.” [Online]. Available: https://nextjs.org/docs (accessed Oct. 15, 2026).

[25] Meta Open Source, “React documentation.” [Online]. Available: https://react.dev (accessed Oct. 15, 2026).

[26] Drizzle Team, “Drizzle ORM documentation.” [Online]. Available: https://orm.drizzle.team/docs/overview (accessed Oct. 15, 2026).

[27] Supabase, “Server-side auth,” *Supabase Documentation*. [Online]. Available: https://supabase.com/docs/guides/auth/server-side (accessed Oct. 15, 2026).

[28] Supabase, “Realtime,” *Supabase Documentation*. [Online]. Available: https://supabase.com/docs/guides/realtime (accessed Oct. 15, 2026).

[29] Supabase, “Connect to your database,” *Supabase Documentation*. [Online]. Available: https://supabase.com/docs/guides/database/connecting-to-postgres (accessed Oct. 15, 2026).

[30] Vitest, “Vitest documentation.” [Online]. Available: https://vitest.dev (accessed Oct. 15, 2026).

[31] Microsoft, “Playwright documentation.” [Online]. Available: https://playwright.dev/docs/intro (accessed Oct. 15, 2026).

[32] World Wide Web Consortium, “Web Content Accessibility Guidelines (WCAG) 2.2,” W3C Recommendation, 2023. [Online]. Available: https://www.w3.org/TR/WCAG22/ (accessed Oct. 15, 2026).

11. APPENDICES

11.1 Selected Source Code

This appendix reproduces the parts of the source code that implement the central rules discussed in Chapter 6. The code is taken from the repository; a few long statements have been wrapped or shortened to fit the page, without changing their logic.

11.1.1 src/lib/server/bookings.ts — reserving and releasing seats

```
/** Atomically reserve seats. Throws (rolling back the transaction) if any tier lacks
capacity. */
export async function reserveInventory(tx: Tx, eventId: string, items: { tierId: string;
quantity: number }[]) {
  // Sorted order prevents deadlocks between concurrent multi-tier bookings.
  for (const item of [...items].sort((a, b) => a.tierId.localeCompare(b.tierId))) {
    const updated = await tx
      .update(ticketTiers)
      .set({ allocated: sql`${ticketTiers.allocated} + ${item.quantity}` })
      .where(
        and(
          eq(ticketTiers.id, item.tierId),
          eq(ticketTiers.eventId, eventId),
          sql`${ticketTiers.allocated} + ${item.quantity} <= ${ticketTiers.capacity}`,
        ),
      )
      .returning({ id: ticketTiers.id });
    if (updated.length === 0) {
      throw new DomainError("Sorry — not enough tickets left in that tier.", "sold_out");
    }
  }
}

/** Compare-and-set status update. Returns the updated row or null if the booking moved
on. */
export async function transitionBooking(tx: DbOrTx, bookingId: string,
    from: BookingStatus | BookingStatus[], to: BookingStatus, patch: Partial<typeof
bookings.$inferInsert> = {}) {
  const fromList = Array.isArray(from) ? from : [from];
  fromList.forEach((f) => assertBookingTransition(f, to));
  const [row] = await tx
    .update(bookings)
    .set({ ...patch, status: to, updatedAt: new Date() })
    .where(and(eq(bookings.id, bookingId), inArray(bookings.status, fromList)))
    .returning();
```

```
  return row ?? null;
}
```

11.1.2 src/lib/domain/eligibility.ts — the audience rule

```
/**
 * Server-authoritative audience check. Uses only the stored profile —
 * never values sent by the client — so request tampering cannot widen access.
 */
export function checkEligibility(event: Audience, user: Viewer): EligibilityResult {
  if (!event.eligibleInstitutionTypes.includes(user.institutionType)) {
    const who = user.institutionType === "SRM" ? "SRM members" : user.institutionType ===
"GUEST" ? "guests" : "students from other colleges";
    return { ok: false, reason: `This event isn't open to ${who}.` };
  }
  if (user.institutionType === "SRM") {
    if (!user.campus || !event.eligibleCampuses.includes(user.campus)) {
      return { ok: false, reason: `This event is limited to
${campusList(event.eligibleCampuses)}.` };
    }
  }
  if (user.institutionType === "EXTERNAL_INSTITUTION" && event.eligibleInstitutions.length
> 0) {
    const mine = normaliseInstitution(user.institutionName);
    if (!mine || !event.eligibleInstitutions.some((i) => normaliseInstitution(i) ===
mine)) {
      return { ok: false, reason: `Only approved institutions can register:
${event.eligibleInstitutions.join(", ")}.` };
    }
  }
  if (!event.eligibleCategories.includes(user.category as ParticipantCategory)) {
    return { ok: false, reason: "Your participant category isn't eligible for this event."
};
  }
  return { ok: true };
}
```

11.1.3 src/lib/server/checkin.ts — recording a check-in exactly once

```
  const now = new Date();
  if (now < event.checkinOpensAt || now > event.checkinClosesAt) {
    return {
      status: "outside_window",
      ticket: info,
```

```
      message: now < event.checkinOpensAt ? "Check-in hasn't opened yet." : "Check-in has
closed for this event.",
    };
  }

  // Unique(ticket_id) makes the first scan win, even across devices.
  const [inserted] = await db
    .insert(checkIns)
    .values({ ticketId: row.ticket.id, eventId, scannedBy: actor.id, method })
    .onConflictDoNothing({ target: checkIns.ticketId })
    .returning();

  if (inserted) {
    await audit(db, { actorId: actor.id, action: "checkin.accepted", entityType: "ticket",
      entityId: row.ticket.id, eventId, details: { ticketNumber: row.ticket.ticketNumber,
method } });
    return { status: "checked_in", ticket: info, scannedAt:
inserted.scannedAt.toISOString() };
  }
```

11.2 Additional Screenshots

The following screenshots show further pages captured during the walkthrough with fictional demonstration data.

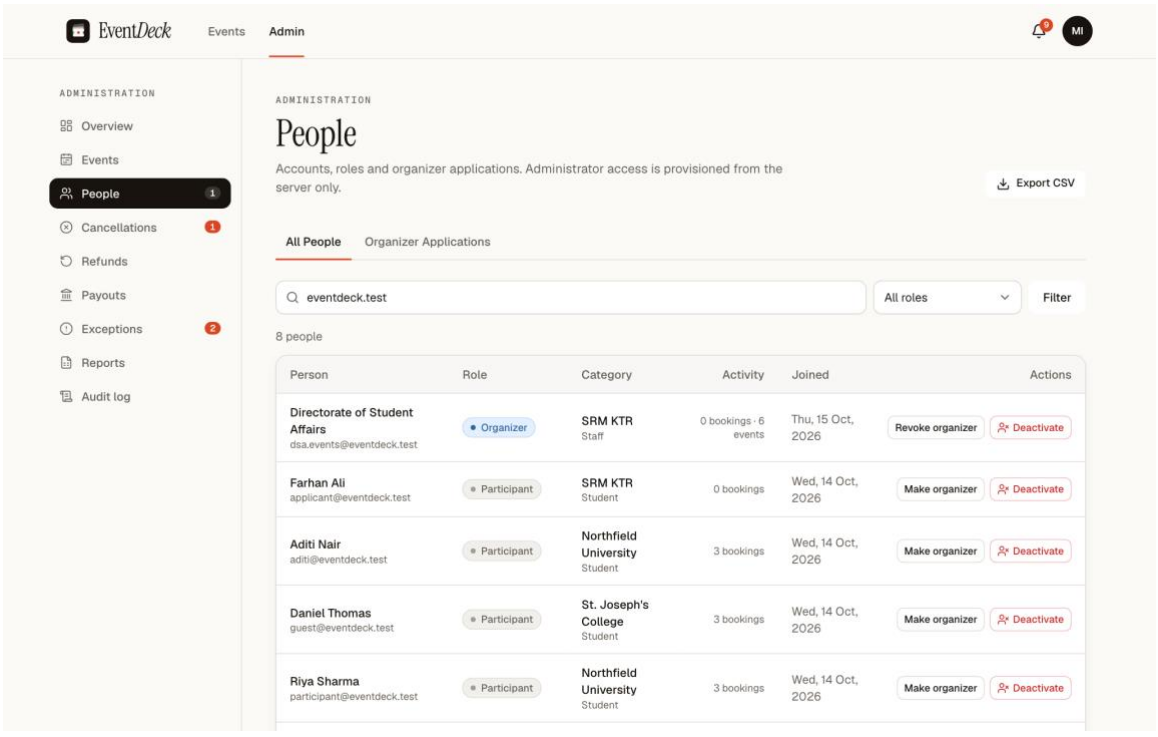


Figure 11.1 People Page Filtered to Demonstration Accounts

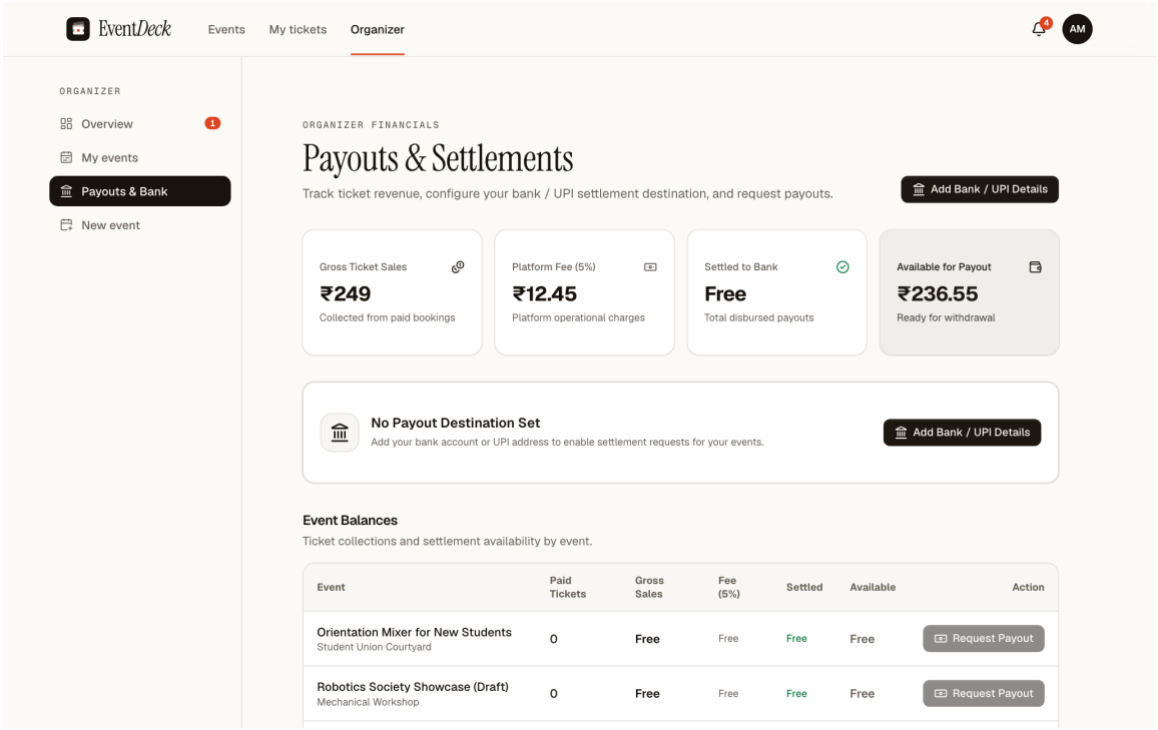


Figure 11.2 Organizer Payouts and Settlements Page

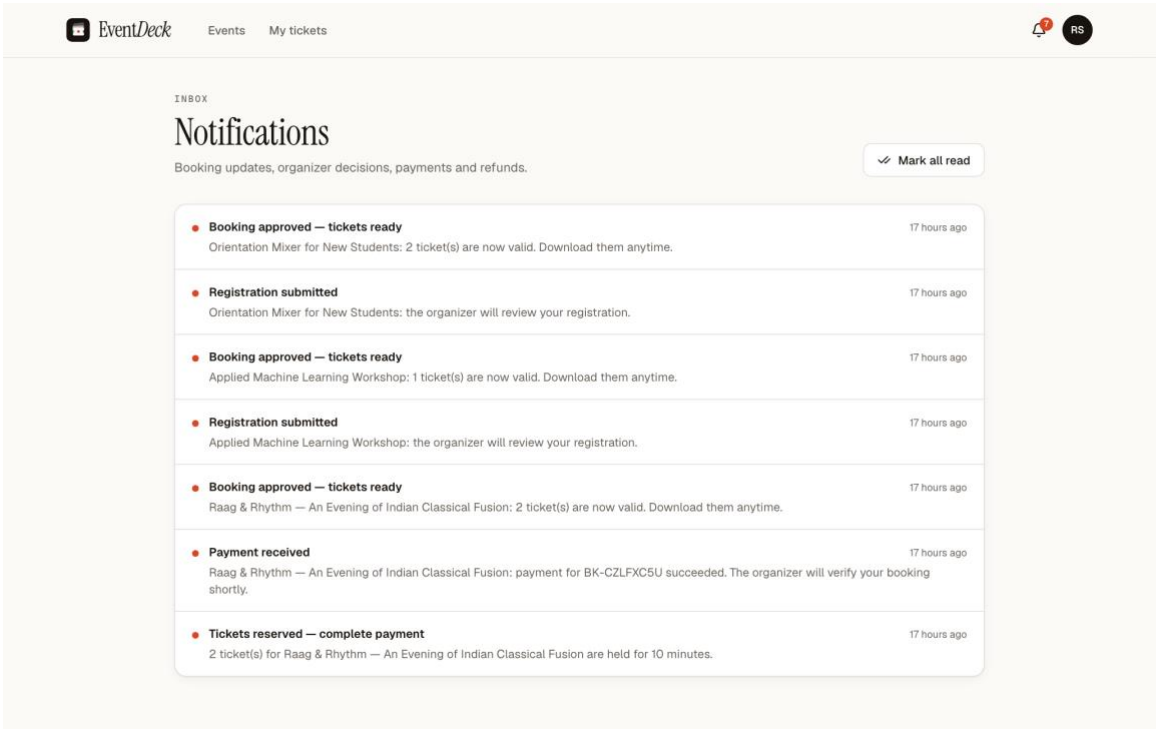


Figure 11.3 Notifications Inbox of a Participant

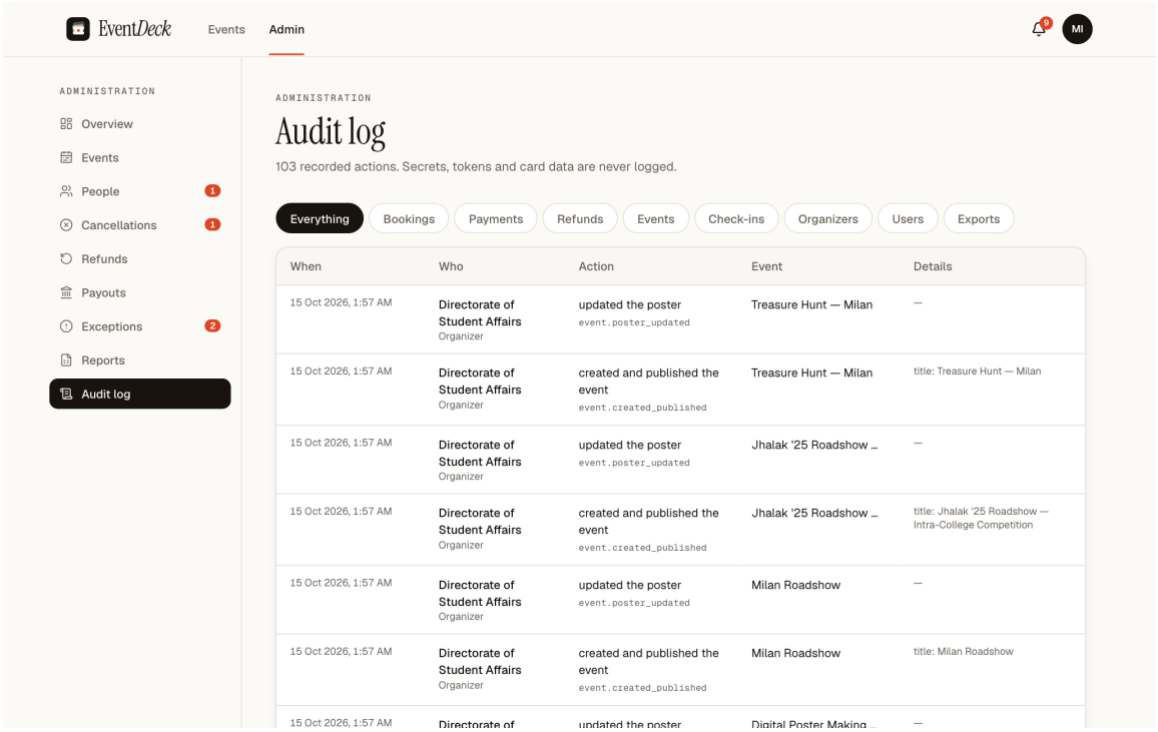


Figure 11.4 Audit Log Viewed by an Administrator

11.3 Automated Test Results

Table 11.1 lists every automated test in the reported run (15 October 2026, 02:06 IST, command npx vitest run --project unit), grouped by file. All 59 tests passed.

Table 11.1 Complete List of Automated Tests and Results

No.	Suite	Test	Result
1	eligibility	event audience eligibility › open events accept every audience	Passed
2	eligibility	event audience eligibility › campus-only events accept only that campus	Passed
3	eligibility	event audience eligibility › inter-campus events accept only the selected campuses	Passed
4	eligibility	event audience eligibility › inter-college events accept only approved institutions (normalised names)	Passed
5	eligibility	event audience eligibility › category rules still apply on top of institution rules	Passed
6	eligibility	event audience eligibility › labels audiences accurately	Passed

No.	Suite	Test	Result
7	eligibility	event audience eligibility › parses approved-institution lists and normalises names	Passed
8	eligibility	event audience eligibility › booking requests cannot smuggle profile fields past validation	Passed
9	eligibility	platform limit: 4 tickets per person › event validation refuses a per-person limit above 4	Passed
10	domain	booking state machine › allows only the documented transitions	Passed
11	domain	booking state machine › terminal states have no exits	Passed
12	domain	booking state machine › tickets are valid only when every gate passes	Passed
13	domain	server-side pricing › prices from database rows and merges duplicate lines	Passed
14	domain	server-side pricing › rejects empty, oversized, negative, fractional and unknown lines	Passed
15	domain	tickets & QR › generates unguessable, unique tokens	Passed
16	domain	tickets & QR › parses QR payloads, raw tokens and typed ticket numbers; rejects junk	Passed
17	domain	tickets & QR › derives readable event codes and unique slugs	Passed
18	domain	CSV export safety › neutralises spreadsheet formulas and quotes properly	Passed
19	domain	campus time & money formatting › round-trips datetime-local values in IST	Passed
20	domain	campus time & money formatting › formats paise as rupees	Passed
21	domain	validation schemas › accepts a well-formed event and coerces numbers	Passed
22	domain	validation schemas › enforces time ordering, unique tier names and price precision	Passed
23	domain	validation schemas › signup never accepts a role and requires a reasonable password	Passed

No.	Suite	Test	Result
24	domain	validation schemas › enforces SRM organization email for host students and allows personal email for external students	Passed
25	auth-onboarding	Participant Registration Validation › validates SRM KTR student registration (requires SRM email & KTR campus)	Passed
26	auth-onboarding	Participant Registration Validation › validates SRM KTR faculty registration (allows SRM or personal email)	Passed
27	auth-onboarding	Participant Registration Validation › validates SRM Ramapuram student registration	Passed
28	auth-onboarding	Participant Registration Validation › validates SRM Vadapalani staff registration (requires official email)	Passed
29	auth-onboarding	Participant Registration Validation › validates Other SRM Campus alumni registration (allows personal email)	Passed
30	auth-onboarding	Participant Registration Validation › validates External college student registration (allows personal email, requires college & city)	Passed
31	auth-onboarding	Participant Registration Validation › validates External college faculty registration	Passed
32	auth-onboarding	Participant Registration Validation › validates External staff registration	Passed
33	auth-onboarding	Participant Registration Validation › validates Guest registration (no college name, campus, or university email required)	Passed
34	auth-onboarding	Participant Registration Validation › rejects mismatched passwords during registration	Passed
35	auth-onboarding	Participant Registration Validation › never accepts client-submitted application role during registration	Passed
36	auth-onboarding	Common Login Validation › requires valid email and non-empty password	Passed
37	auth-onboarding	Common Login Validation › normalizes login email to lowercase	Passed

No.	Suite	Test	Result
38	auth-onboarding	Role-Based Authorization & Dashboard Routing › maps the three application roles to the correct dashboards	Passed
39	payouts	payout validation schemas › payoutAccountSchema › validates a valid Indian bank account	Passed
40	payouts	payout validation schemas › payoutAccountSchema › rejects mismatched account numbers	Passed
41	payouts	payout validation schemas › payoutAccountSchema › rejects invalid IFSC formats	Passed
42	payouts	payout validation schemas › payoutAccountSchema › validates a valid UPI ID	Passed
43	payouts	payout validation schemas › payoutAccountSchema › rejects invalid UPI IDs	Passed
44	payouts	payout validation schemas › requestPayoutSchema › accepts valid payout request for an event	Passed
45	payouts	payout validation schemas › requestPayoutSchema › rejects payout below minimum threshold or non-uuid event	Passed
46	payouts	payout validation schemas › processPayoutSchema › approves payout with UTR number	Passed
47	payouts	payout validation schemas › processPayoutSchema › requires valid uuid for payoutId	Passed
48	payouts	payout validation schemas › fee calculations › computes 5% platform fee correctly	Passed
49	security	Razorpay signature verification › accepts a correct checkout signature and rejects tampering	Passed
50	security	Razorpay signature verification › verifies webhooks against the exact raw body	Passed
51	security	payment mode guard › never runs live keys and never runs the simulator in production	Passed
52	security	poster upload type sniffing › trusts magic bytes, not file names	Passed

No.	Suite	Test	Result
53	url	URL resolution and auth redirect helper › always uses permanent production URL on Vercel production deployment	Passed
54	url	URL resolution and auth redirect helper › uses permanent production URL when NEXT_PUBLIC_VERCEL_ENV is production	Passed
55	url	URL resolution and auth redirect helper › normalizes temporary .vercel.app URLs to production URL in NODE_ENV=production	Passed
56	url	URL resolution and auth redirect helper › respects custom non-vercel production domain when NODE_ENV=production	Passed
57	url	URL resolution and auth redirect helper › respects local development URL and trims trailing slashes	Passed
58	url	URL resolution and auth redirect helper › falls back to localhost:3000 when no environment variables are set in development	Passed
59	url	URL resolution and auth redirect helper › formats VERCEL_URL properly for preview deployments when outside production	Passed

11.4 Demonstration Data Used for Screenshots

All names, institutions and events in the screenshots are fictional demonstration data created by the project's seed scripts, except for the SRM Directorate of Student Affairs posters, which were supplied by the project team to illustrate real campus events and are shown with fictional dates and bookings. Passwords are not reproduced here.

Table 11.2 Fictional Demonstration Data

Account or item	Role / details	Used in
Riya Sharma (participant@eventdeck.test)	Participant; two approved bookings including a paid Gold booking	Figures 6.3, 7.1, 11.3

Account or item	Role / details	Used in
Dr. Arjun Mehta (organizer@eventdeck.test)	Organizer of HackNorth 2026 and other technical events	Figures 5.2, 5.3, 5.8, 6.5, 6.6, 7.2, 7.3, 11.2
Kavya Rao (cultural@eventdeck.test)	Organizer of Raag & Rhythm and other cultural events	Figure 7.4
Meera Iyer (admin@eventdeck.test)	Administrator	Figures 7.5–7.7, 11.1, 11.4
Directorate of Student Affairs (dsa.events@eventdeck.test)	Organizer of six poster events (Rendezvous, Namma Chennai, Milan, Jhalak and others)	Figures 5.5–5.7
Aditi Nair, Daniel Thomas, Farhan Ali	Further participants and an organizer applicant	Figures 7.4, 11.1
Events	Sixteen demonstration events dated October 2026 to February 2027	Chapters 5–7

11.5 Glossary

Table 11.3 Glossary of Terms

Term	Meaning
Allocated seats	The number of seats of a tier currently held by live bookings.
Audience rules	Event settings that limit booking by institution type, SRM campus, approved college and participant category.
Compare-and-set	An update that succeeds only if the row is still in the expected state.
Complimentary pass	A free ticket requested with a reason and approved by the organizer.
Hold	The ten-minute reservation of seats for a paid booking awaiting payment.
Idempotency key	A unique value sent with a request so that repeating the request has no further effect.
Moderator	An account assigned to one event to scan tickets at the entrance.
Payment exception	A verified payment that could not be applied to a booking, left for an administrator.

Term	Meaning
QR token	A 192-bit random value that is the only content of a ticket's QR code.
Row-level security (RLS)	Database policies that limit which rows each user can read or change.
Server Action	A server function in Next.js invoked directly from a form.
Tier	A category of ticket within an event, with its own price and capacity.
UTR	Unique Transaction Reference of a bank transfer, recorded when a payout is completed.
Webhook	A signed HTTP request sent by Razorpay to report payment and refund events.